

Современные методы анализа данных и машинного обучения

Тема 11. Лекция 13

Глубинное обучение. Техники глубинного обучения

Юрий Саночкин

ysanochkin@hse.ru

НИУ ВШЭ, 2026

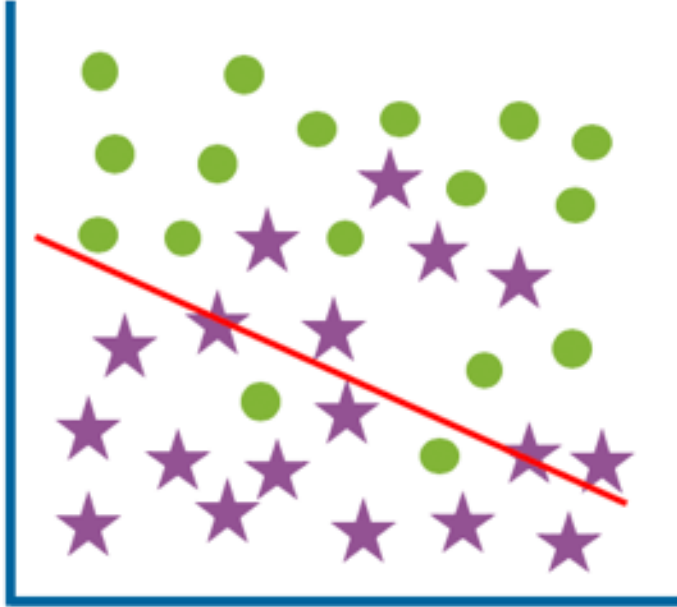
Нейронные сети

- На протяжении последних встреч мы с вами часто употребляли слово «переобучение».
- «Нейронные сети имеют сильные тенденции к переобучению.»

Нейронные сети

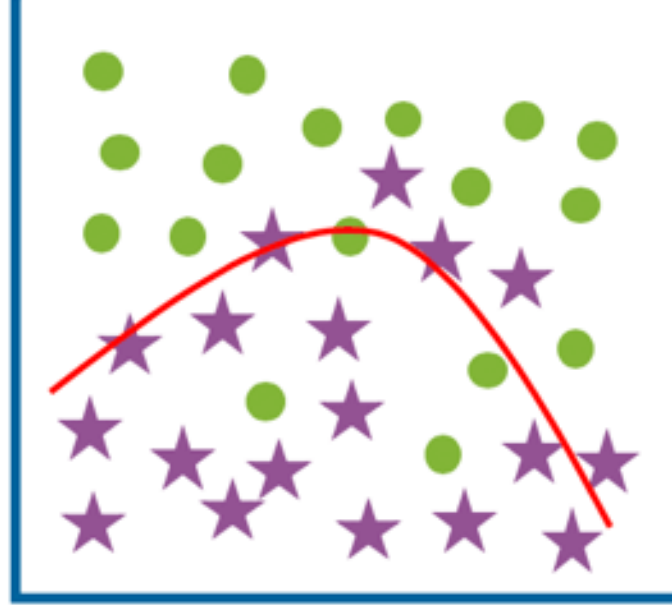
- На протяжении последних встреч мы с вами часто употребляли слово «переобучение».
- «Нейронные сети имеют сильные тенденции к переобучению.»
- Напомните, а что вообще такое переобучение?

Underfit
(high bias)



High training error
High test error

Optimum



Low training error
Low test error

Overfit
(high variance)



Low training error
High test error

Переобучение нейронных сетей

- Переобучение (overfitting) — это результат чрезмерной подгонки параметров модели к зависимостям, содержащимся в обучающем множестве.

Переобучение нейронных сетей

- Переобучение (overfitting) — это результат чрезмерной подгонки параметров модели к зависимостям, содержащимся в обучающем множестве.
- Говоря более простым языком — модель начинает сдвигаться в сторону «запоминания» тренировочной выборки, вместо попыток поиска в ней настоящих зависимостей.

Переобучение нейронных сетей

- Переобучение (overfitting) — это результат чрезмерной подгонки параметров модели к зависимостям, содержащимся в обучающем множестве.
- Говоря более простым языком — модель начинает сдвигаться в сторону «запоминания» тренировочной выборки, вместо попыток поиска в ней настоящих зависимостей.
- Переобученные модели характеризуются (на тренировочной выборке) низким bias (смещение) и высокой variance (дисперсия) — это классические компоненты, на которые может быть разложена любая ошибка.

Переобучение нейронных сетей

- Какие могут быть причины переобучения?

Переобучение нейронных сетей

- Какие могут быть причины переобучения?
- Смещение в данных

Переобучение нейронных сетей

- Какие могут быть причины переобучения?
- Смещение в данных
 - Несбалансированность классов

Переобучение нейронных сетей

- Какие могут быть причины переобучения?
- Смещение в данных
 - Несбалансированность классов
 - Много шума

Переобучение нейронных сетей

- Какие могут быть причины переобучения?
- Смещение в данных
 - Несбалансированность классов
 - Много шума
 - Выбросы

Переобучение нейронных сетей

- Какие могут быть причины переобучения?
- Смещение в данных
 - Несбалансированность классов
 - Много шума
 - Выбросы
 - Разное распределение обучающей и тестовой выборки

Переобучение нейронных сетей

- Какие могут быть причины переобучения?
- Смещение в данных
 - Несбалансированность классов
 - Много шума
 - Выбросы
 - Разное распределение обучающей и тестовой выборки
- Недостаток данных

Переобучение нейронных сетей

- Какие могут быть причины переобучения?
- Смещение в данных
 - Несбалансированность классов
 - Много шума
 - Выбросы
 - Разное распределение обучающей и тестовой выборки
- Недостаток данных
- Избыток параметров

Переобучение нейронных сетей

- Какие могут быть причины переобучения?
- Смещение в данных
 - Несбалансированность классов
 - Много шума
 - Выбросы
 - Разное распределение обучающей и тестовой выборки
- Недостаток данных
- Избыток параметров

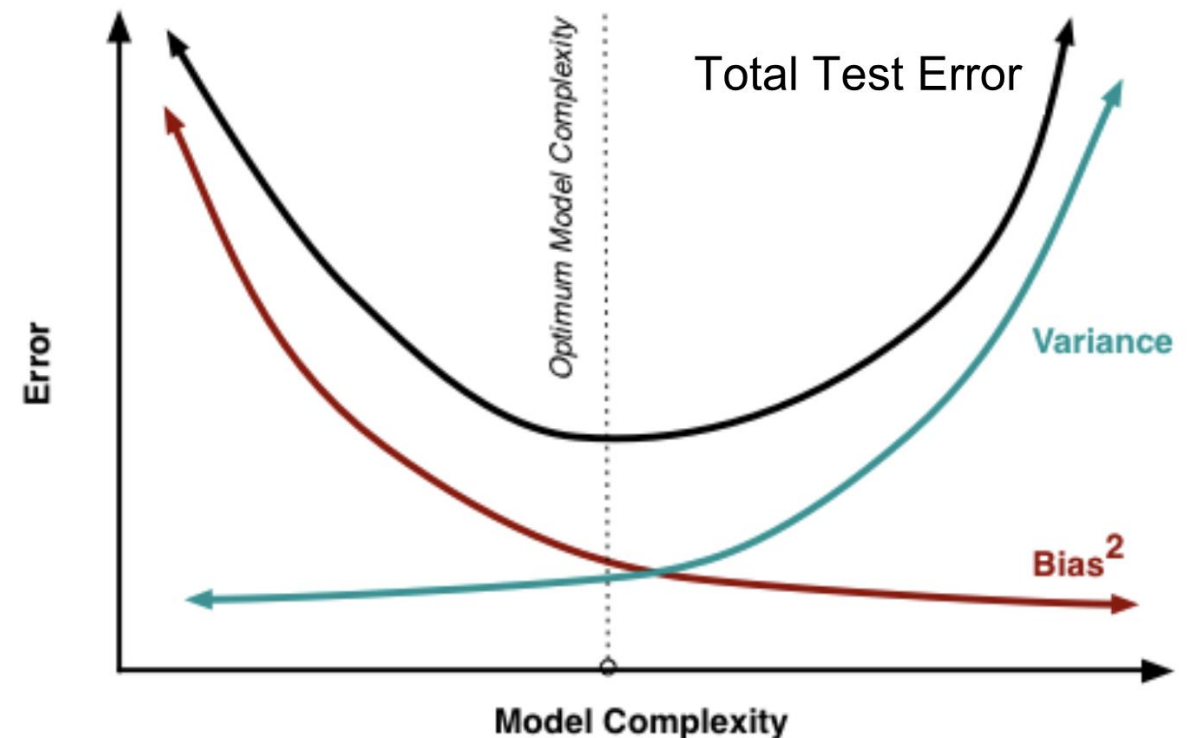
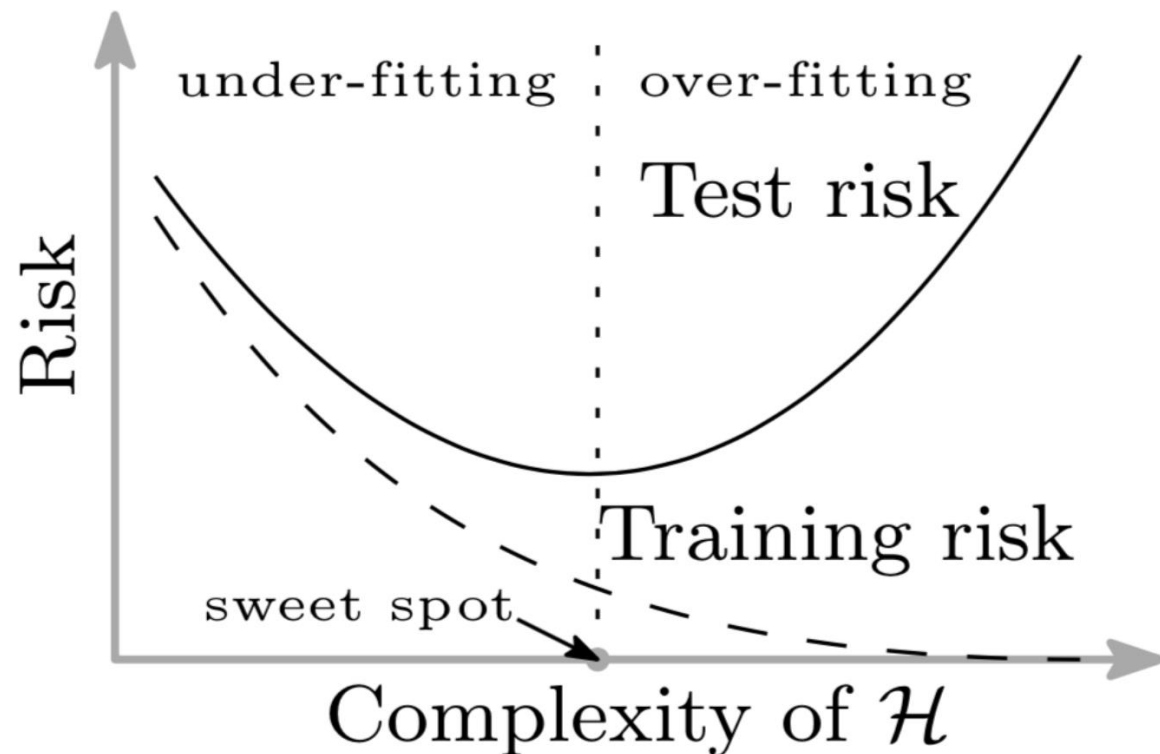
Стоп, а разве последнее — это плохо?! Что плохого в большом количестве параметров — это ведь просто означает, что модель сложная, а значит, сможет найти сложные зависимости! Супер же, разве нет?!...

Переобучение нейронных сетей

- Классическая теория анализа bias-variance модели говорит нам о следующем:

Переобучение нейронных сетей

- Классическая теория анализа bias-variance модели говорит нам о следующем:



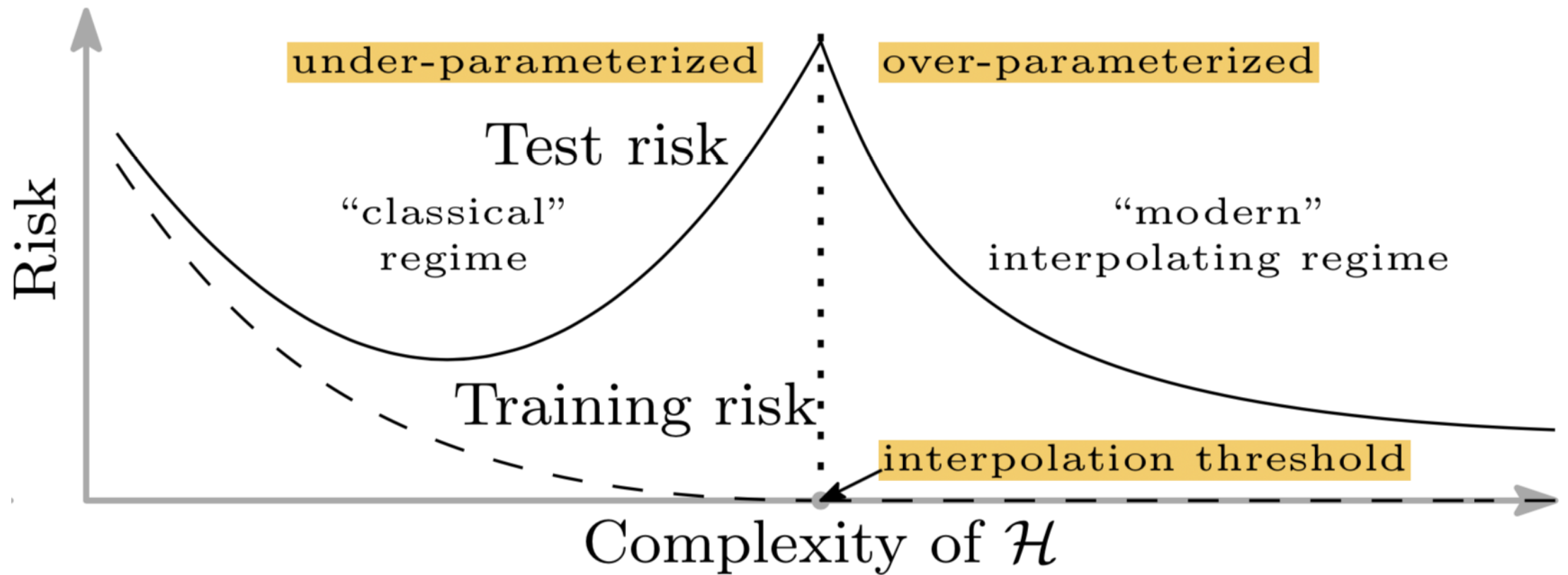
Переобучение нейронных сетей

- Однако последние исследования показывают, что в реальности — по крайней мере для глубинного обучения — это может быть не вполне верно.

Переобучение нейронных сетей

- Однако последние исследования показывают, что в реальности — по крайней мере для глубинного обучения — это может быть не вполне верно.
- Михаил Белкин с коллегами в своей статье 2018 года доказывают, что теоретическая кривая bias-variance trade-off для моделей глубинного обучения будет иметь так называемую double-U форму — вместо классической, давно известной и представленной нами на предыдущей визуализации U-образной формы.

Переобучение нейронных сетей



Переобучение нейронных сетей

- Впрочем, несмотря на существенное математическое обоснование полученных авторами выводов, а также несмотря на значительный резонанс в сообществе после выхода их статьи, — в реальности воспроизвести данные результаты эмпирически — достаточно сложно (хотя и возможно).

Переобучение нейронных сетей

- Впрочем, несмотря на существенное математическое обоснование полученных авторами выводов, а также несмотря на значительный резонанс в сообществе после выхода их статьи, — в реальности воспроизвести данные результаты эмпирически — достаточно сложно (хотя и возможно).
- Связано это главным образом с большим количеством параметров, после которого может происходить подобный «излом» кривой bias-variance, а также с рядом других аспектов, которые в обязательном порядке должны быть учтены при построении модели, для того чтобы описанная в статье концепция могла полноценно работать.

Переобучение нейронных сетей

- На практике же для нас это означает, что, хотя, согласно теории, чем больше параметров у вашей модели глубинного обучения (читай — чем более сложная нейросеть), тем, с какого-то момента, лучше должно получаться качество её предсказаний, — в действительности же это вполне может и не работать, а переобучение вашей нейронной сети может легко оказаться даже хуже, чем у какой-нибудь сложной модели классического машинного обучения.

Переобучение нейронных сетей

- На практике же для нас это означает, что, хотя, согласно теории, чем больше параметров у вашей модели глубинного обучения (читай — чем более сложная нейросеть), тем, с какого-то момента, лучше должно получаться качество её предсказаний, — в действительности же это вполне может и не работать, а переобучение вашей нейронной сети может легко оказаться даже хуже, чем у какой-нибудь сложной модели классического машинного обучения.
- И вот ровно по этой самой причине первая подтема нашей сегодняшней лекции — это регуляризация нейронных сетей — или, другими словами, способы борьбы с переобучением в данных моделях.

Регуляризация нейронных сетей

Регуляризация нейронных сетей

```
graph TD; A[Регуляризация нейронных сетей] --> B[Через функцию потерь]; A --> C[Изменение структуры сети]; A --> D[Изменение данных]; B --> B1["• L1-регуляризация"]; B --> B2["• L2-регуляризация"]; C --> C1["• Batchnorm"]; C --> C2["• Dropout"]; D --> D1["• Аугментации"];
```

Через функцию потерь

- L_1 -регуляризация
- L_2 -регуляризация

Изменение структуры сети

- Batchnorm
- Dropout

Изменение данных

- Аугментации

Регуляризация через функцию потерь

- Регуляризация через функцию потерь — это класс методов регуляризаций, позволяющих уменьшить сложность модели за счёт добавления штрафных слагаемых к её функции потерь.

Регуляризация через функцию потерь

- Регуляризация через функцию потерь — это класс методов регуляризаций, позволяющих уменьшить сложность модели за счёт добавления штрафных слагаемых к её функции потерь.
- Наиболее распространенные методы из этого класса известны как регуляризация L_1 и L_2 .
- Обе они также широко применяются и в классическом машинном обучении — например, для модели линейной регрессии.

Регуляризация через функцию потерь

- Общая формула выглядит следующим образом:

$$\min_f \left[\sum_i \mathcal{L}(f(x_i), y_i) + \lambda R(f) \right]$$

Регуляризация через функцию потерь

- Общая формула выглядит следующим образом:

$$\min_f \left[\sum_i \mathcal{L}(f(x_i), y_i) + \lambda R(f) \right]$$

L_1 -регуляризация

L_2 -регуляризация

$$R(f(x|w)) = \|w\|_1 = \sum_j |w_j|$$
$$\min_{f,w} \left[\sum_i \mathcal{L}(f(x_i), y_i) + \lambda \sum_j |w_j| \right]$$

$$R(f(x|w)) = \|w\|_2 = \sum_j w_j^2$$
$$\min_{f,w} \left[\sum_i \mathcal{L}(f(x_i), y_i) + \lambda \sum_j w_j^2 \right]$$

Регуляризация через изменение данных

- Регуляризация через изменение данных — это класс методов регуляризаций, позволяющих препятствовать переобучению модели за счёт расширения набора обучающих данных.

Регуляризация через изменение данных

- Регуляризация через изменение данных — это класс методов регуляризаций, позволяющих препятствовать переобучению модели за счёт расширения набора обучающих данных.
- Осуществляется это как правило путём изменений исходных изображений. Такой процесс называется аугментация.

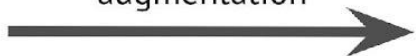
Регуляризация через изменение данных

- Регуляризация через изменение данных — это класс методов регуляризаций, позволяющих препятствовать переобучению модели за счёт расширения набора обучающих данных.
- Осуществляется это как правило путём изменений исходных изображений. Такой процесс называется аугментация.
- В рамках аугментаций применяются следующие операции:
 - Повороты
 - Отражения
 - Масштабирования
 - Изменения яркости,
 - Добавление шума и т.д.

Original image



augmentation



Horizontal Flip



Crop



Median Blur



Contrast



Hue / Saturation / Value



Gamma



Регуляризация через изменение структуры сети

- Регуляризация через изменение структуры сети — это класс методов регуляризаций, позволяющих препятствовать переобучению модели за счёт временной или постоянной модификации внутренней структуры нейронной сети.

Регуляризация через изменение структуры сети

- Регуляризация через изменение структуры сети — это класс методов регуляризаций, позволяющих препятствовать переобучению модели за счёт временной или постоянной модификации внутренней структуры нейронной сети.
- Среди подходов этого класса выделим два основных метода: батч-нормализацию (batchnorm) и дропаут (dropout).

Батч-нормализация

- Основная цель слоя батч-нормализации — а это полноценный слой в нейронной сети — корректировать распределение признаков во время обучения вашей модели.
- Для признака размерности $[B, H, W, C]$ считается C средних и C выборочных дисперсий.

Батч-нормализация

- Основная цель слоя батч-нормализации — а это полноценный слой в нейронной сети — корректировать распределение признаков во время обучения вашей модели.
- Для признака размерности $[B, H, W, C]$ считается C средних и C выборочных дисперсий.

За что тут отвечает каждая из букв? Почему их четыре?
И почему именно C средних и C выборочных дисперсий?

Батч-нормализация

- В каждом слое батч-нормализации есть два статистических параметра: среднее и выборочная дисперсия — и два обучаемых: γ и β .
- Если статистические параметры неудачно нормализуют признаки, то обучаемые параметры позволят сети самой подкорректировать интенсивность и сдвиг признаков на выходе.

Батч-нормализация

- В каждом слое батч-нормализации есть два статистических параметра: среднее и выборочная дисперсия — и два обучаемых: γ и β .
- Если статистические параметры неудачно нормализуют признаки, то обучаемые параметры позволят сети самой подкорректировать интенсивность и сдвиг признаков на выходе.
- При необходимости, сеть может даже выучить параметры γ и β таким образом, чтобы на выходе получались исходные признаки без нормализации вообще.

Батч-нормализация

- Также важно запомнить, что во время инференса среднее и выборочная дисперсия не рассчитываются — используются их экспоненциально сглаженные оценки, которые считаются во время тренировки.

Батч-нормализация

- Вход:

Батч-нормализация

- Вход:
 - Значения признака (предыдущего выхода) x для пакета (батча)
 $B = \{x_i\}_{i=1}^m$

Батч-нормализация

- Вход:
 - Значения признака (предыдущего выхода) x для пакета (батча)
 $B = \{x_i\}_{i=1}^m$;
 - Настраиваемые (обучаемые) параметры γ, β ;
 - Константа ε для вычислительной устойчивости.

Батч-нормализация

- Вход:
 - Значения признака (предыдущего выхода) x для пакета (батча)
 $B = \{x_i\}_{i=1}^m$;
 - Настраиваемые (обучаемые) параметры γ, β ;
 - Константа ε для вычислительной устойчивости.
- Выход:

Батч-нормализация

- Вход:
 - Значения признака (предыдущего выхода) x для пакета (батча)
 $B = \{x_i\}_{i=1}^m$;
 - Настраиваемые (обучаемые) параметры γ, β ;
 - Константа ε для вычислительной устойчивости.
- Выход:
 - $\{y_i = BN_{\gamma, \beta}(x_i)\}_{i=1}^m$

Батч-нормализация по шагам

Батч-нормализация по шагам

1. Расчет статистических параметров по батчу:

Батч-нормализация по шагам

1. Расчет статистических параметров по батчу:

- Выборочное среднее по батчу:

$$\overline{x_B} = \frac{1}{m} \sum_{i=1}^m x_i$$

Батч-нормализация по шагам

1. Расчет статистических параметров по батчу:

- Выборочное среднее по батчу:

$$\overline{x_B} = \frac{1}{m} \sum_{i=1}^m x_i$$

- Выборочная дисперсия по батчу:

$$S_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \overline{x_B})^2$$

Батч-нормализация по шагам

2. Стандартизация входного слоя:

Батч-нормализация по шагам

2. Стандартизация входного слоя:

$$\hat{x}_i = \frac{x_i - \overline{x_B}}{\sqrt{S_B^2 + \varepsilon}}$$

Батч-нормализация по шагам

2. Стандартизация входного слоя:

$$\hat{x}_i = \frac{x_i - \overline{x_B}}{\sqrt{S_B^2 + \varepsilon}}$$

3. Сжатие и сдвиг:

Батч-нормализация по шагам

2. Стандартизация входного слоя:

$$\hat{x}_i = \frac{x_i - \overline{x_B}}{\sqrt{S_B^2 + \varepsilon}}$$

3. Сжатие и сдвиг:

$$y_i = \gamma \hat{x}_i + \beta$$

Батч-нормализация по шагам

2. Стандартизация входного слоя:

$$\hat{x}_i = \frac{x_i - \overline{x_B}}{\sqrt{S_B^2 + \varepsilon}}$$

3. Сжатие и сдвиг:

$$y_i = \gamma \hat{x}_i + \beta \equiv BN_{\gamma, \beta}(x_i)$$

Батч-нормализация по шагам

2. Стандартизация входного слоя:

$$\hat{x}_i = \frac{x_i - \overline{x_B}}{\sqrt{S_B^2 + \varepsilon}}$$

3. Сжатие и сдвиг:

$$y_i = \gamma \hat{x}_i + \beta \equiv BN_{\gamma, \beta}(x_i)$$

- Обратим внимание на то, что если $\beta = \overline{x_B}$, а $\gamma = \sqrt{S_B^2 + \varepsilon}$, то из этого следует, что $y_i = x_i$, о возможности чего мы говорили ранее. Параметры β и γ настраиваются в процессе обучения.

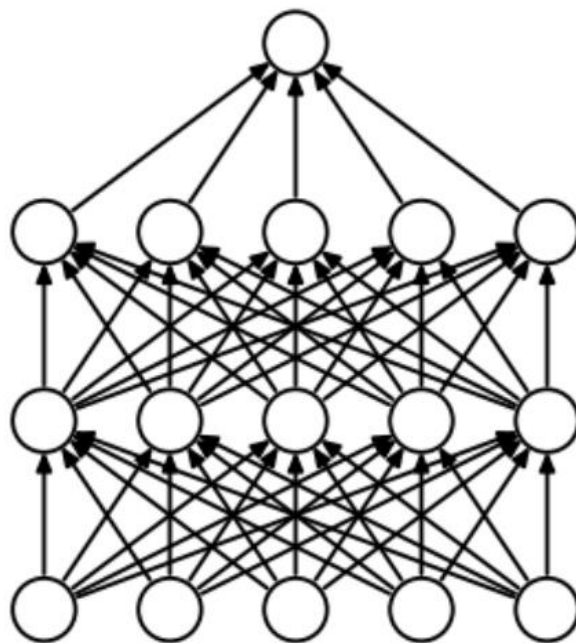
Батч-нормализация



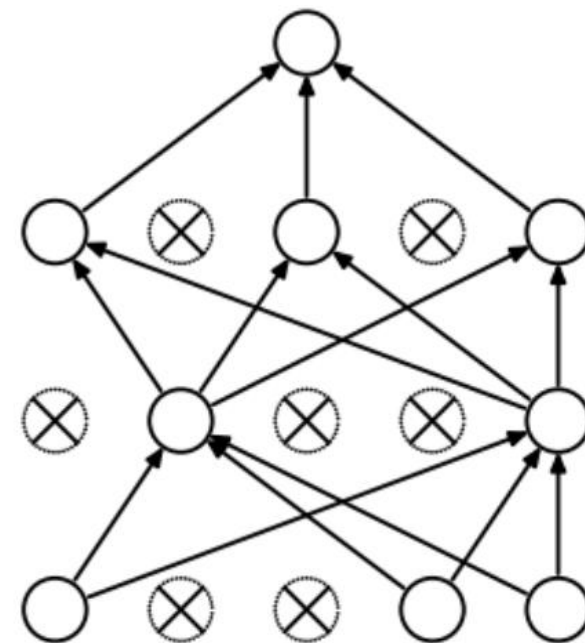
Dropout

Dropout

- В процессе обучения сети мы можем временно отключать некоторые нейроны.
- Это уменьшит обобщающую способность сети на каждом конкретном примере, но сохранит количество параметров.



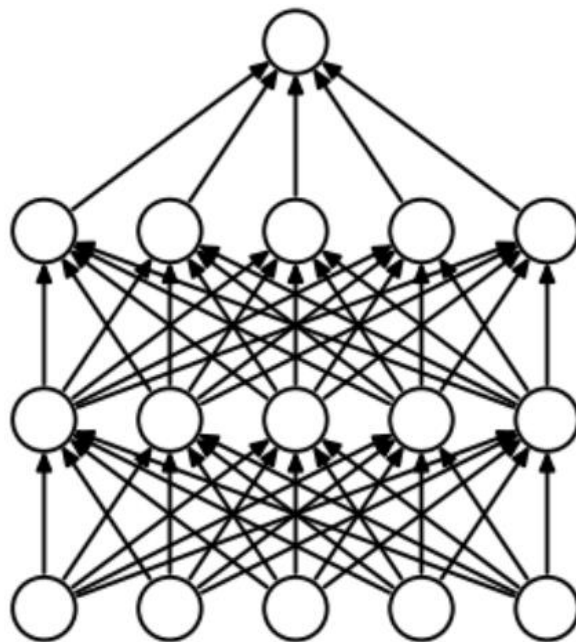
(a) Standard Neural Net



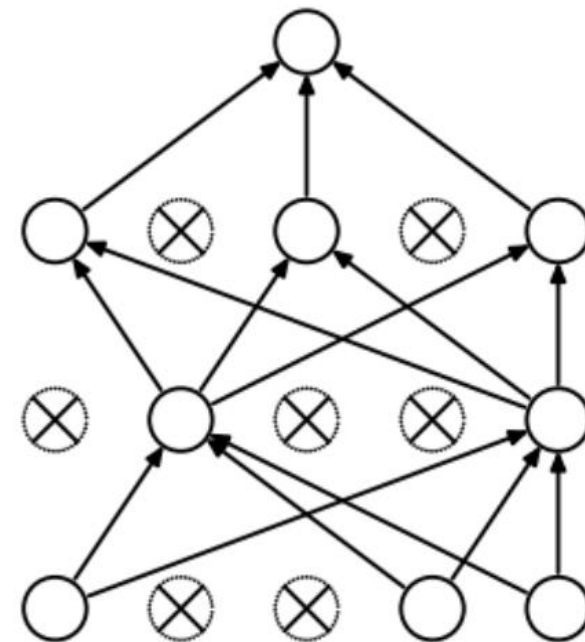
(b) After applying dropout.

Dropout

- При «выключении» некоторых нейронов произойдёт резкое изменение предсказаний модели, что приведёт к увеличению ошибки, а полученные градиенты этой ошибки укажут, как её исправить с использованием оставшихся нейронов.



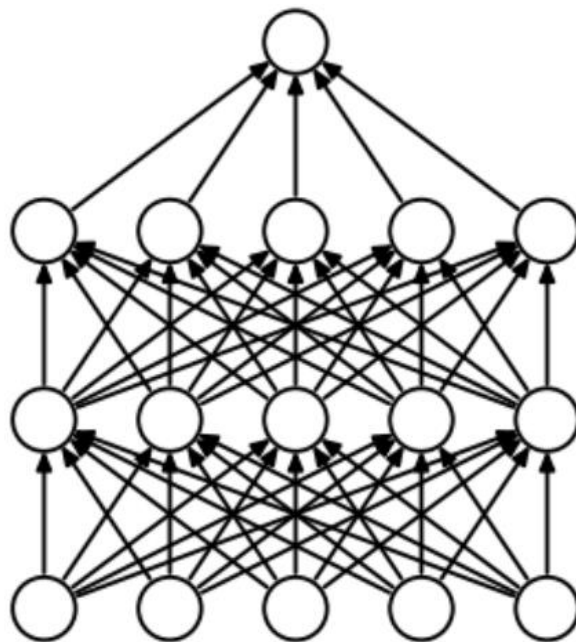
(a) Standard Neural Net



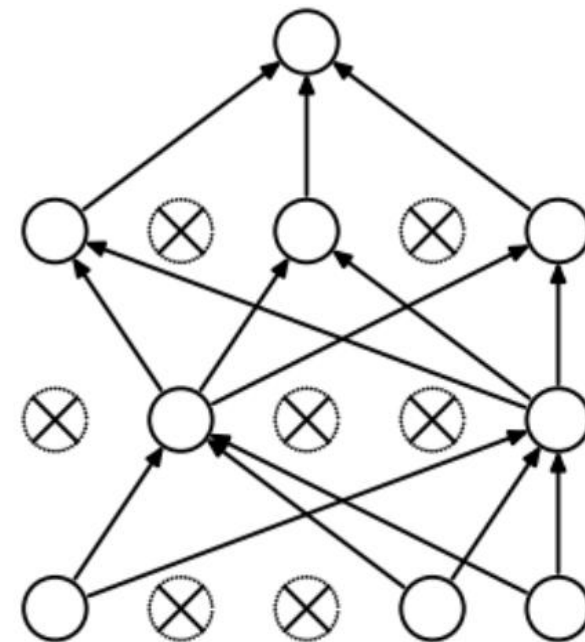
(b) After applying dropout.

Dropout

- Такая техника регуляризации называется Dropout и на многих задачах она показывает свою крайне высокую эффективность.



(a) Standard Neural Net



(b) After applying dropout.

Dropout

- Реализация метода проста: слой Dropout — а это тоже полноценный слой нейронной сети — умножает выход предыдущего слоя на маску из нулей и единиц.

Dropout

- Реализация метода проста: слой Dropout — а это тоже полноценный слой нейронной сети — умножает выход предыдущего слоя на маску из нулей и единиц.
 - $x^{k+1} = x^k \odot \text{mask}$

Dropout

- Реализация метода проста: слой Dropout — а это тоже полноценный слой нейронной сети — умножает выход предыдущего слоя на маску из нулей и единиц.
 - $x^{k+1} = x^k \odot \text{mask}$
- Данная маска участвует и в подсчете градиентов.

Dropout

- Реализация метода проста: слой Dropout — а это тоже полноценный слой нейронной сети — умножает выход предыдущего слоя на маску из нулей и единиц.
 - $x^{k+1} = x^k \odot \text{mask}$
- Данная маска участвует и в подсчете градиентов.
 - $\nabla_{x^k} \mathcal{L} = \nabla_{x^{k+1}} \mathcal{L} \odot \text{mask}$

Dropout

- Реализация метода проста: слой Dropout — а это тоже полноценный слой нейронной сети — умножает выход предыдущего слоя на маску из нулей и единиц.
 - $x^{k+1} = x^k \odot \text{mask}$
- Данная маска участвует и в подсчете градиентов.
 - $\nabla_{x^k} \mathcal{L} = \nabla_{x^{k+1}} \mathcal{L} \odot \text{mask}$
- Маска генерируется независимо на каждом шаге прямого прохода сети.

Dropout

- Реализация метода проста: слой Dropout — а это тоже полноценный слой нейронной сети — умножает выход предыдущего слоя на маску из нулей и единиц.
 - $x^{k+1} = x^k \odot \text{mask}$
- Данная маска участвует и в подсчете градиентов.
 - $\nabla_{x^k} \mathcal{L} = \nabla_{x^{k+1}} \mathcal{L} \odot \text{mask}$
- Маска генерируется независимо на каждом шаге прямого прохода сети.
 - $\text{mask}_i \sim \text{Be}(1 - p)$

Dropout

- Реализация метода проста: слой Dropout — а это тоже полноценный слой нейронной сети — умножает выход предыдущего слоя на маску из нулей и единиц.
 - $x^{k+1} = x^k \odot \text{mask}$
- Данная маска участвует и в подсчете градиентов.
 - $\nabla_{x^k} \mathcal{L} = \nabla_{x^{k+1}} \mathcal{L} \odot \text{mask}$
- Маска генерируется независимо на каждом шаге прямого прохода сети.
 - $\text{mask}_i \sim \text{Be}(1 - p)$

Объясните обозначения в последней строке.
Что такое Be ? Что показывает p ?

Dropout

- На этапе предсказания Dropout «выключается»: внутренние представления используются как есть, без умножения на маску.
- Чтобы на этом этапе нейроны не получали дополнительных активаций и не становились «перевозбужденными» из-за того, что они теперь все активны, веса нейронов домножаются на обратную вероятность быть выкинутыми.

Dropout

- На этапе предсказания Dropout «выключается»: внутренние представления используются как есть, без умножения на маску.
- Чтобы на этом этапе нейроны не получали дополнительных активаций и не становились «перевозбужденными» из-за того, что они теперь все активны, веса нейронов домножаются на обратную вероятность быть выкинутыми.
 - $x^{k+1} = (1 - p) \cdot x^k$

Dropout

- Важно отметить, что Dropout-слой не рекомендуется размещать между сверточными слоями.

Dropout

- Важно отметить, что Dropout-слой не рекомендуется размещать между сверточными слоями.
- На это есть две причины:
 - Обычно в сверточных сетях не так много параметров и в начале обучения им нужно как можно меньше обнулений.

Dropout

- Важно отметить, что Dropout-слой не рекомендуется размещать между сверточными слоями.
- На это есть две причины:
 - Обычно в сверточных сетях не так много параметров и в начале обучения им нужно как можно меньше обнулений.
 - Из-за пространственной корреляции карт признаков обнуление отдельных активаций бесполезно.

Dropout

- Важно отметить, что Dropout-слой не рекомендуется размещать между сверточными слоями.
- На это есть две причины:
 - Обычно в сверточных сетях не так много параметров и в начале обучения им нужно как можно меньше обнулений.
 - Из-за пространственной корреляции карт признаков обнуление отдельных активаций бесполезно.
- В большинстве современных сверточных архитектур Dropout заменен на Global Average Pooling.

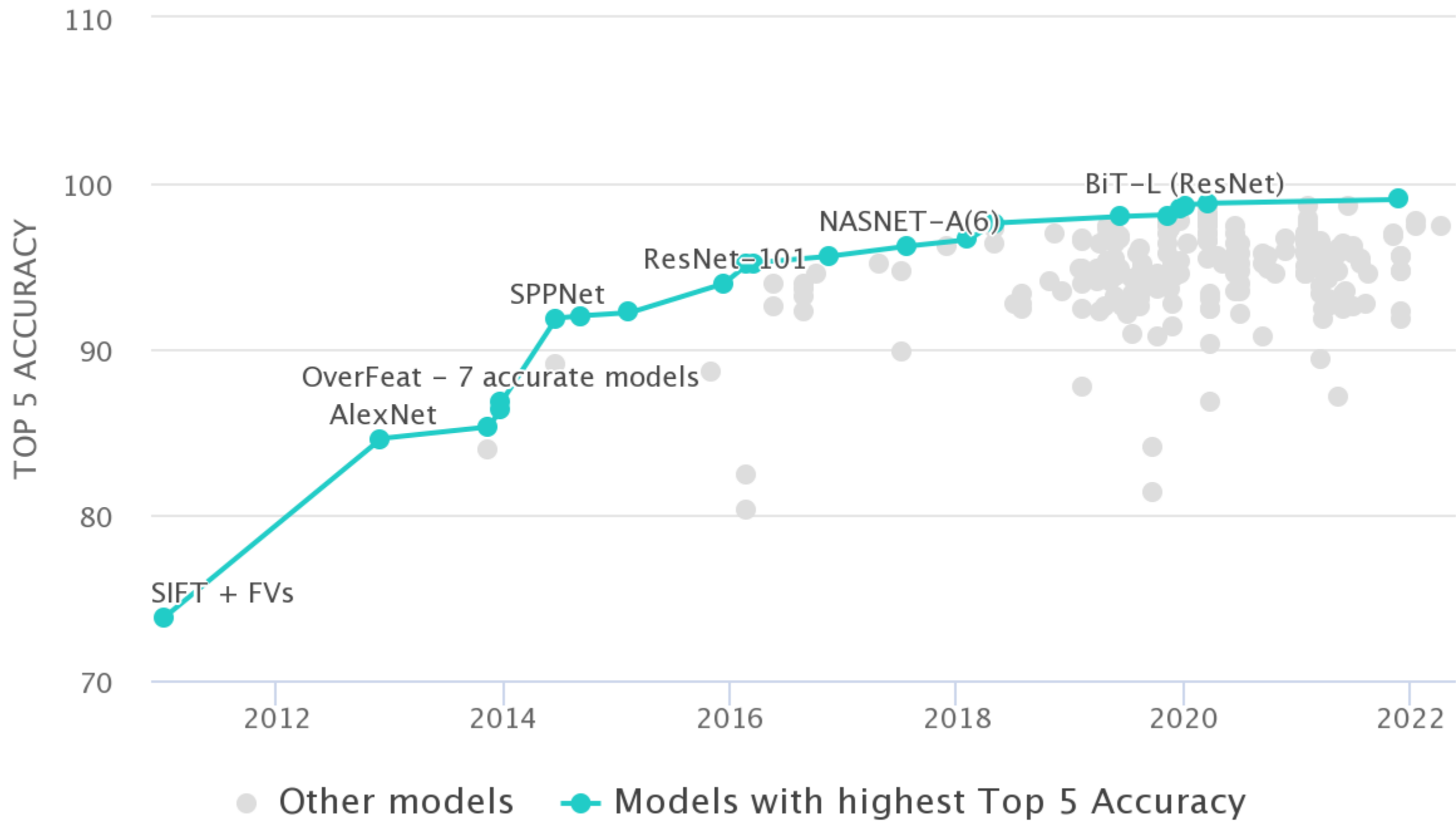
Dropout

- Как взаимодействуют друг с другом Dropout и Batchnorm?
- Еще одна особенность использования Dropout заключается в том, что использовать его стоит после Batchnorm-слоя, а не перед.
- В противном случае Batchnorm-слой будет собирать неверные статистики и это может даже привести к общему негативному эффекту.

Ключевые архитектуры CNN

Ключевые архитектуры CNN

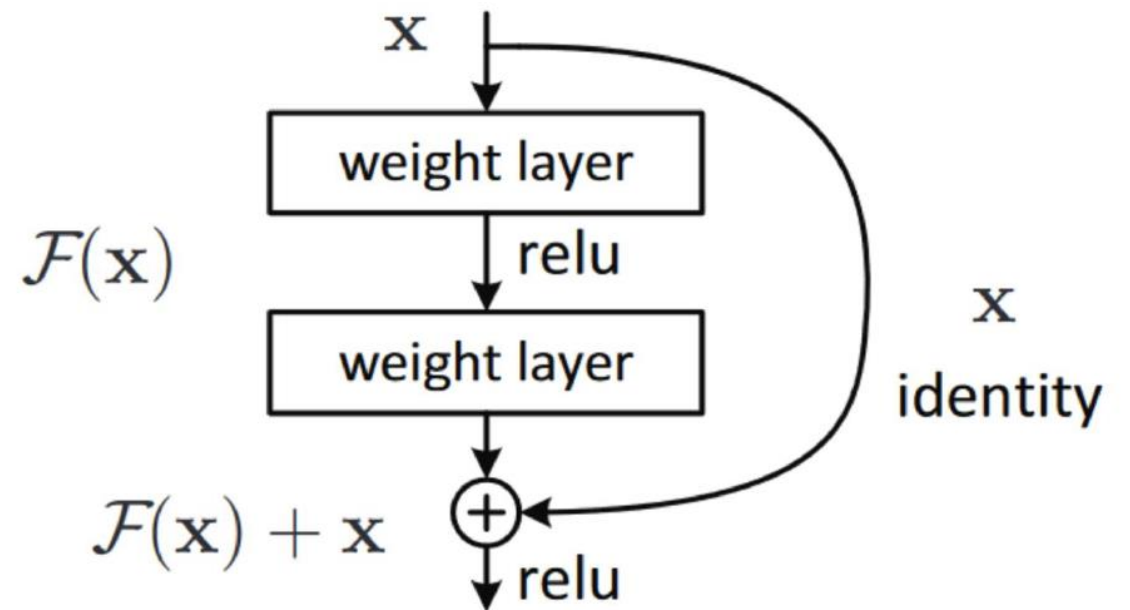
- Теперь, когда мы познакомились с множеством важных и широко распространенных техник глубокого обучения, давайте обсудим несколько наиболее значимых — в том числе с точки зрения влияния на индустрию — семейств архитектур сверточных нейронных сетей.



Семейство ResNet (2015)

Семейство ResNet (2015)

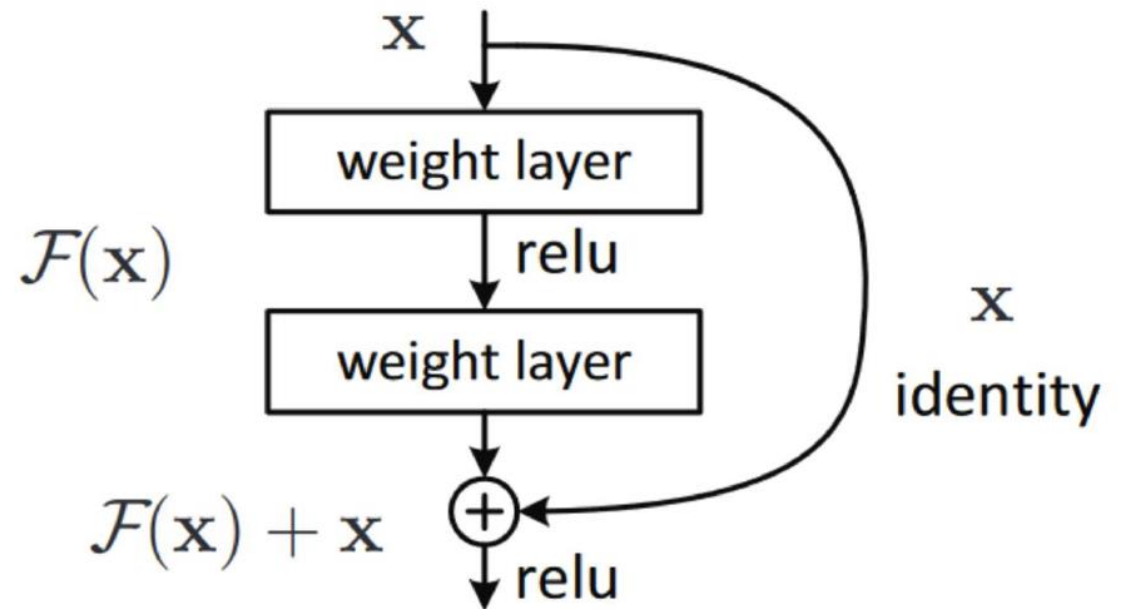
- Основная идея семейства нейронных сетей ResNet сводится к введению так называемого остаточного блока.
- Суть его заключается в следующем.



Семейство ResNet (2015)

- Сверточные слои нейросети аппроксимируют функцию $H(x)$, где x — вход слоя.
- Нейросеть по определению также может выучить residual-функцию (остаточную функцию):

$$F(x) = H(x) - x$$

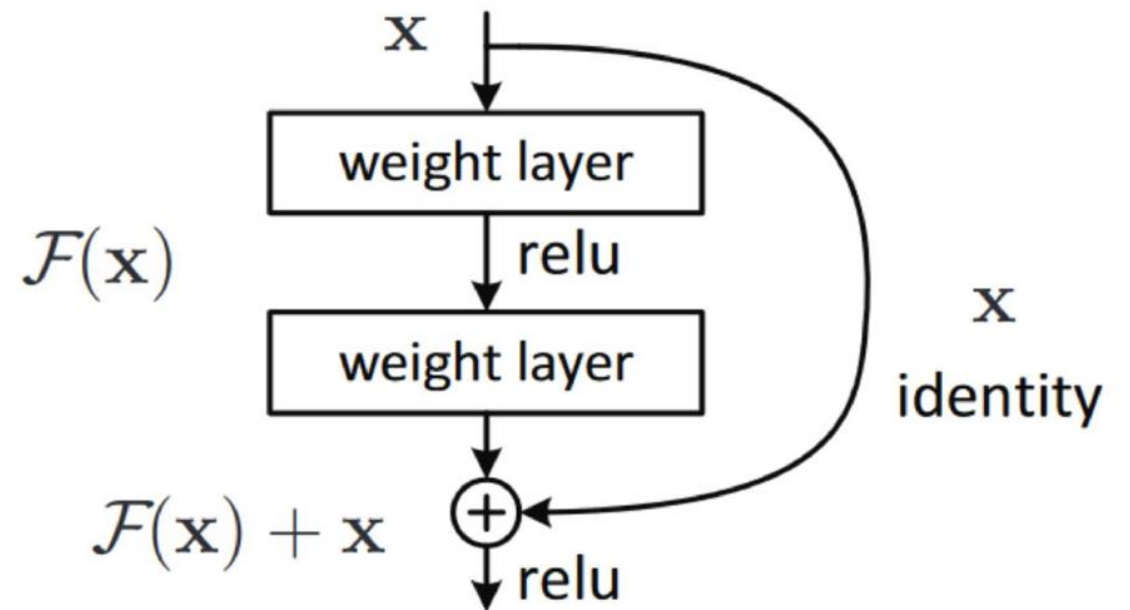


Семейство ResNet (2015)

- Тогда целевую функцию можно переписать следующим образом:

$$H(x) = \mathcal{F}(x) + x$$

- А далее ввести в слои shortcut-соединение x (см. картинку).



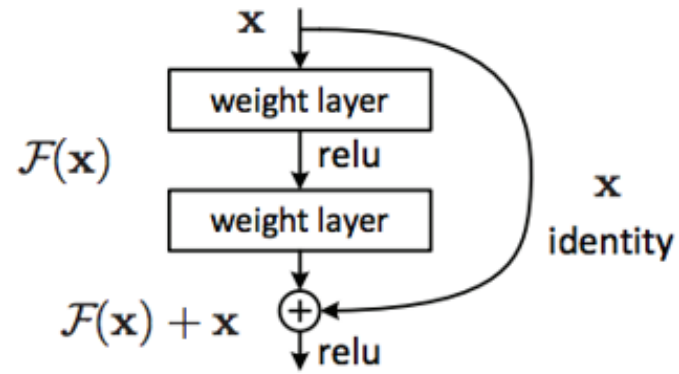
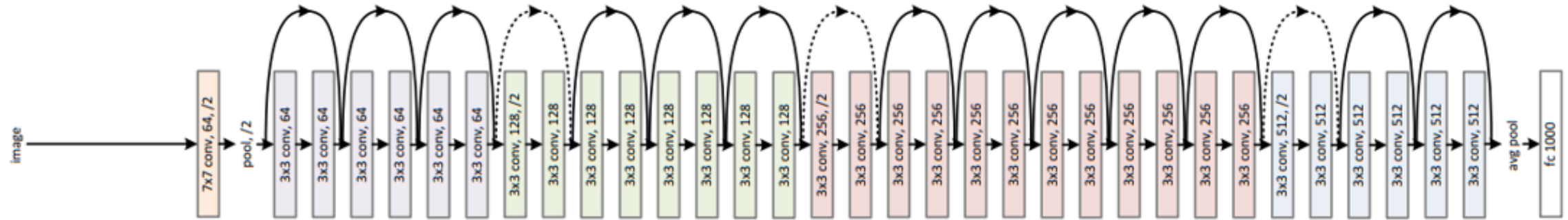
Семейство ResNet (2015)

- С ростом глубины в сверточных нейросетях появились две проблемы:
 - Затухание градиентов;
 - Снижение качества модели в процессе обучения.

Семейство ResNet (2015)

- С ростом глубины в сверточных нейросетях появились две проблемы:
 - Затухание градиентов;
 - Снижение качества модели в процессе обучения.
- Сеть ResNet:
 - Берет за основу идею остаточного обучения (которую мы рассмотрели на предыдущих слайдах);
 - Решает обе проблемы глубоких сверточных нейронных сетей.

Residual Networks (ResNet50)



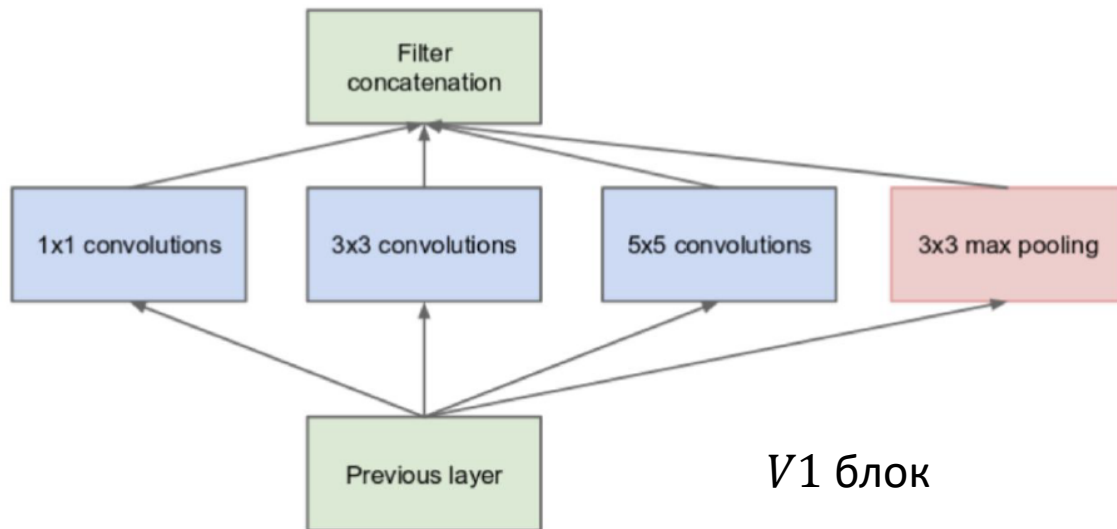
Семейство Inception (2015)

Семейство Inception (2015)

- Основная идея семейства нейронных сетей Inception сводится к введению Inception-блоков.

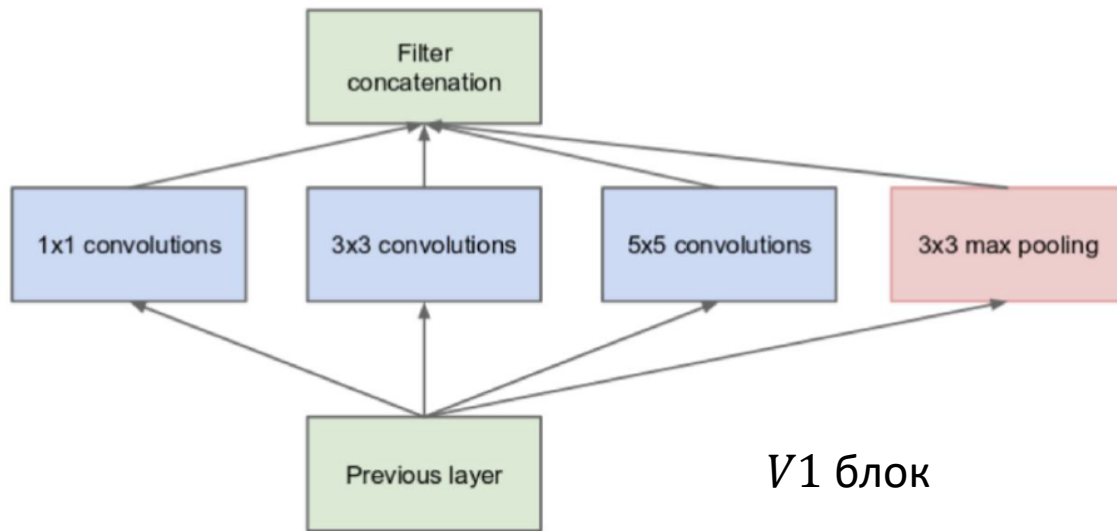
Семейство Inception (2015)

- Основная идея семейства нейронных сетей Inception сводится к введению Inception-блоков.

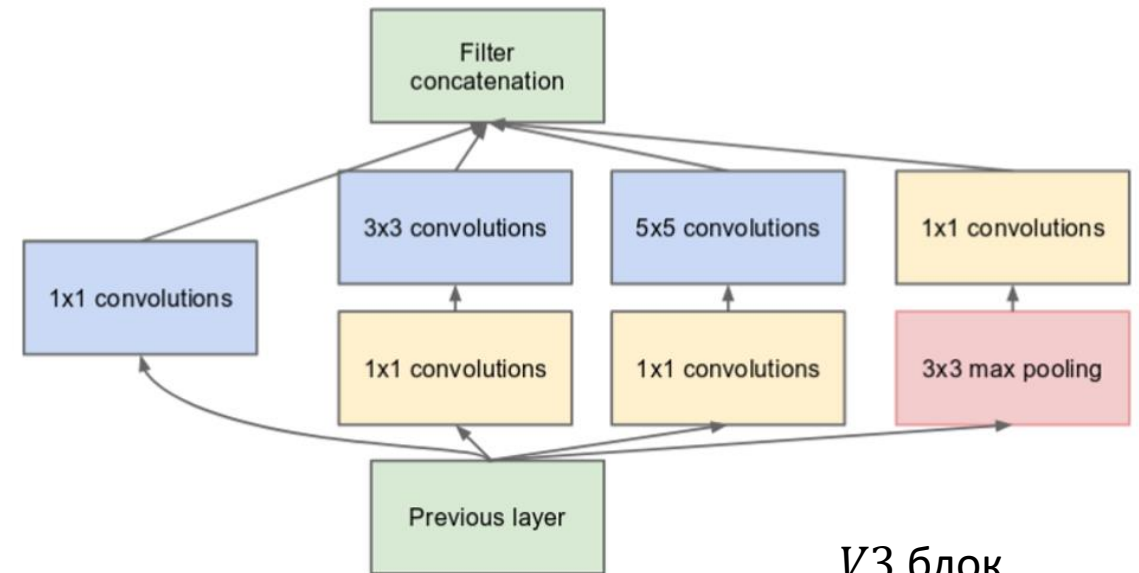


Семейство Inception (2015)

- Основная идея семейства нейронных сетей Inception сводится к введению Inception-блоков.

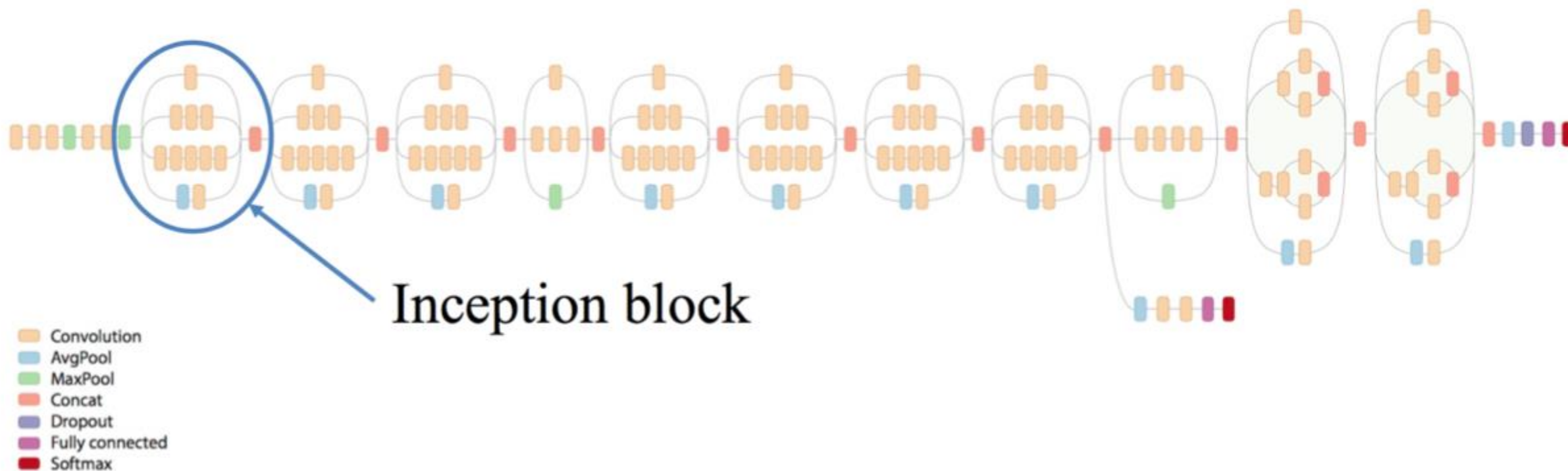


V1 блок

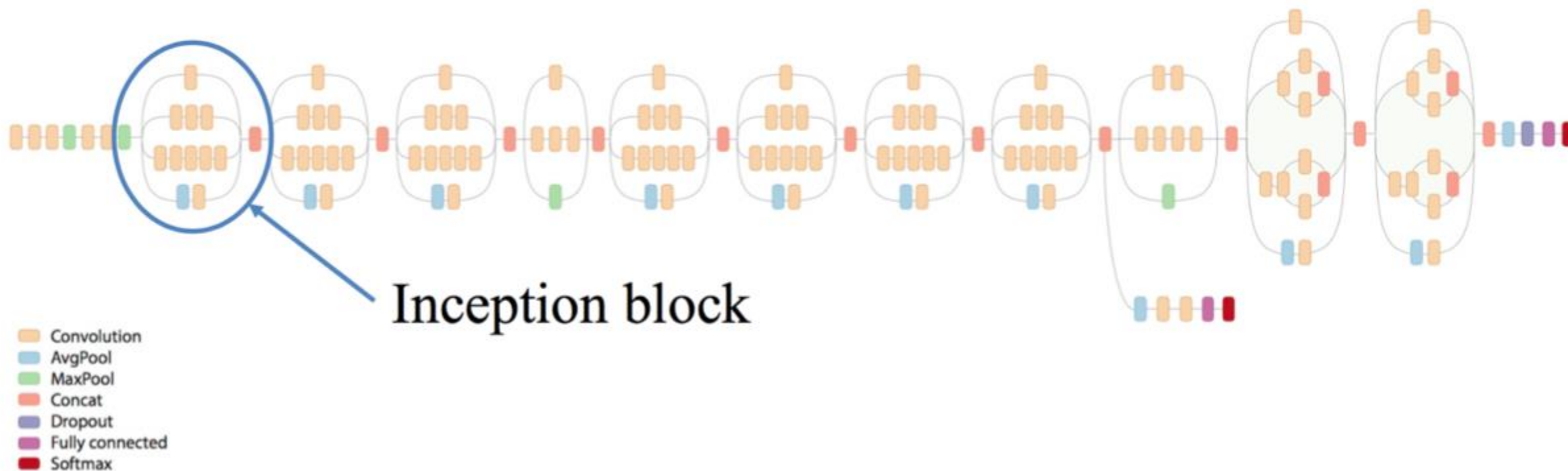


V3 блок

Семейство Inception (2015)



Семейство Inception (2015)



Количество параметров — 23.9 млн
Top-5 Accuracy ImageNet — 93.7%

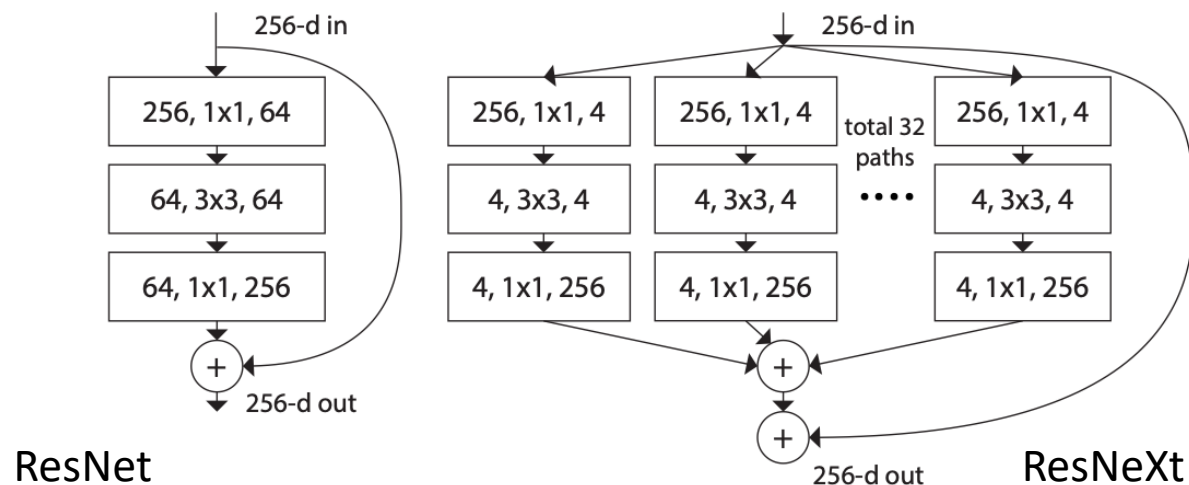
Семейство ResNeXt (2017)

Семейство ResNeXt (2017)

- Семейство ResNeXt объединило предыдущие идеи ResNet и Inception.

Семейство ResNeXt (2017)

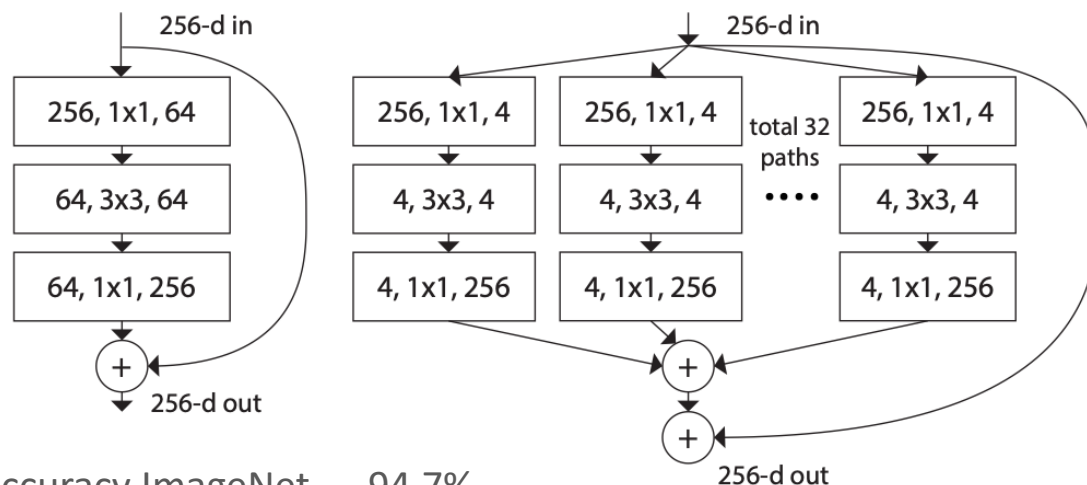
- Семейство ResNeXt объединило предыдущие идеи ResNet и Inception.
- В ResNeXt используется новый гиперпараметр cardinality — количество путей в блоке сети.



stage	output	ResNet-50	ResNeXt-50 ($32 \times 4d$)
conv1	112×112	7×7 , 64, stride 2	7×7 , 64, stride 2
conv2	56×56	3×3 max pool, stride 2	3×3 max pool, stride 2
		$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128, C=32 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3	28×28	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256, C=32 \\ 1 \times 1, 512 \end{bmatrix} \times 4$
conv4	14×14	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512, C=32 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$
conv5	7×7	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 1024 \\ 3 \times 3, 1024, C=32 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	global average pool 1000-d fc, softmax	global average pool 1000-d fc, softmax
# params.		25.5×10^6	25.0×10^6
FLOPs		4.1×10^9	4.2×10^9

Семейство ResNeXt (2017)

- Семейство ResNeXt объединило предыдущие идеи ResNet и Inception.
- В ResNeXt используется новый гиперпараметр cardinality — количество путей в блоке сети.



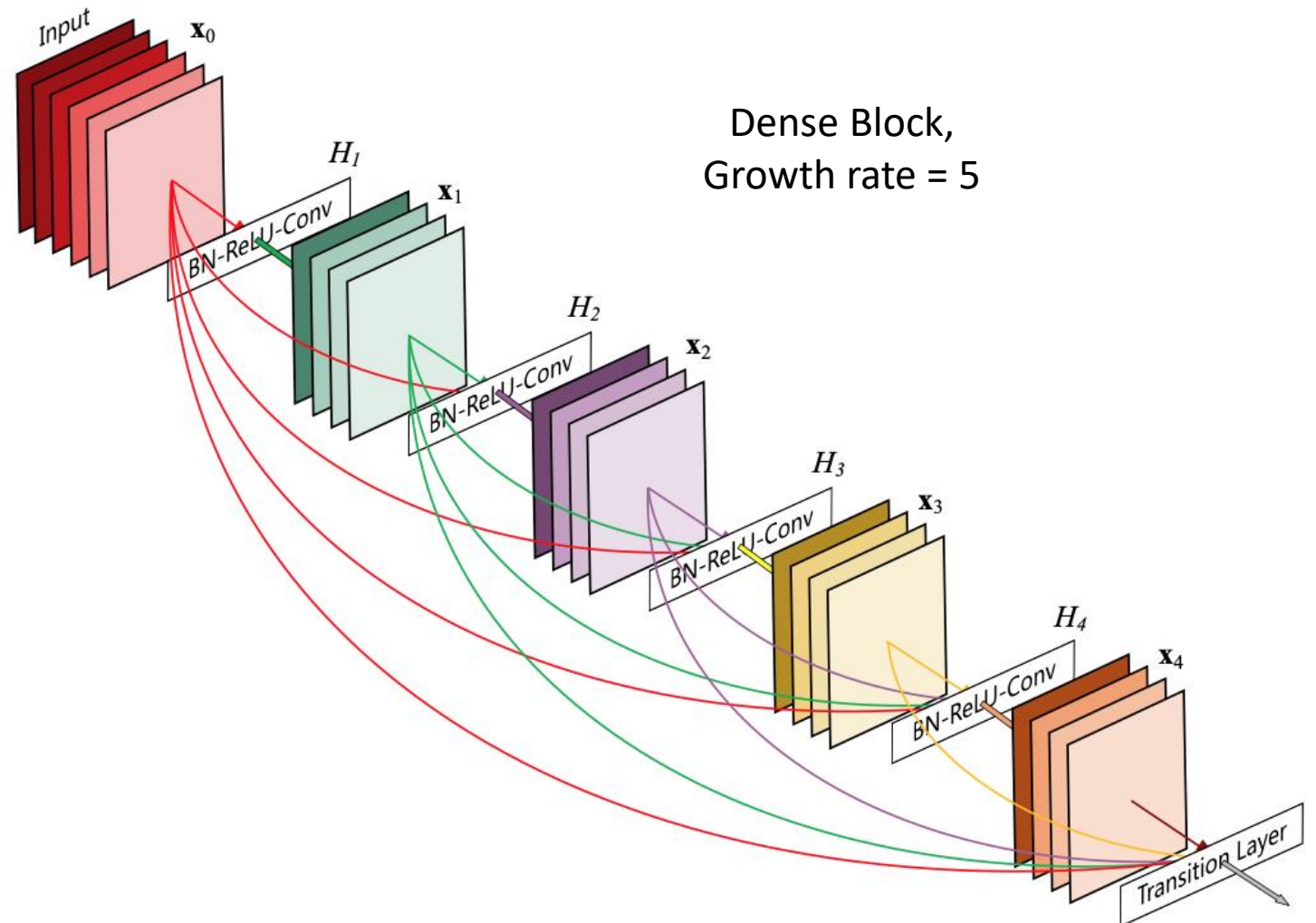
Top-5 Accuracy ImageNet — 94.7%

stage	output	ResNet-50	ResNeXt-50 ($32 \times 4d$)
conv1	112×112	7×7 , 64, stride 2	7×7 , 64, stride 2
conv2	56×56	3×3 max pool, stride 2	3×3 max pool, stride 2
		$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128, C=32 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3	28×28	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256, C=32 \\ 1 \times 1, 512 \end{bmatrix} \times 4$
conv4	14×14	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512, C=32 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$
conv5	7×7	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 1024 \\ 3 \times 3, 1024, C=32 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	global average pool 1000-d fc, softmax	global average pool 1000-d fc, softmax
# params.		25.5×10^6	25.0×10^6
FLOPs		4.1×10^9	4.2×10^9

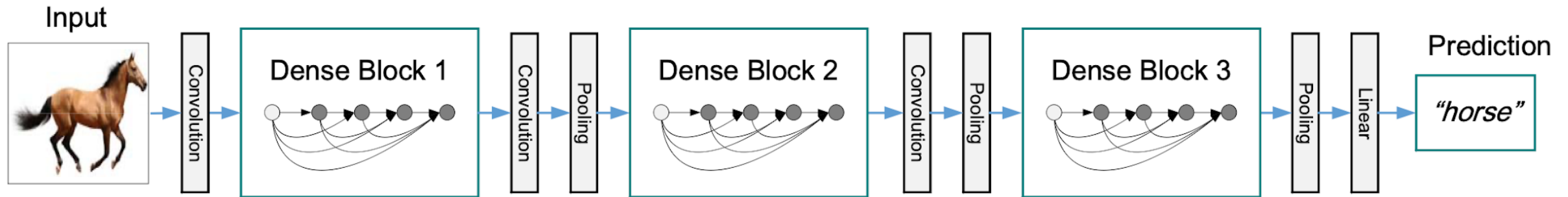
Семейство DenseNet (2018)

Семейство DenseNet (2018)

- Семейство DenseNet ввело идею Dense-блоков.
- В Dense-блоке каждый слой берет на вход все предшествующие карты признаков из этого же блока.



Семейство DenseNet (2018)



Семейство MobileNet (2018)

Семейство MobileNet (2018)

- Первая свёрточная нейронная сеть, работающая на мобильных устройствах.

Семейство MobileNet (2018)

- Первая свёрточная нейронная сеть, работающая на мобильных устройствах.

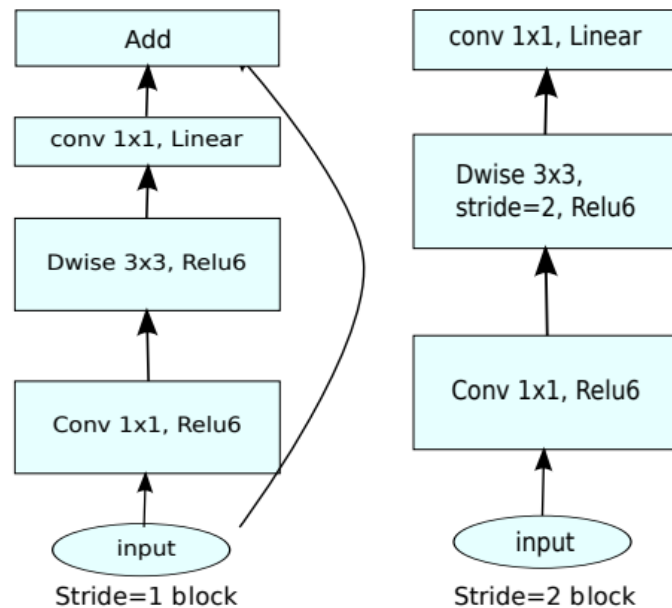


Table 1. MobileNet Body Architecture

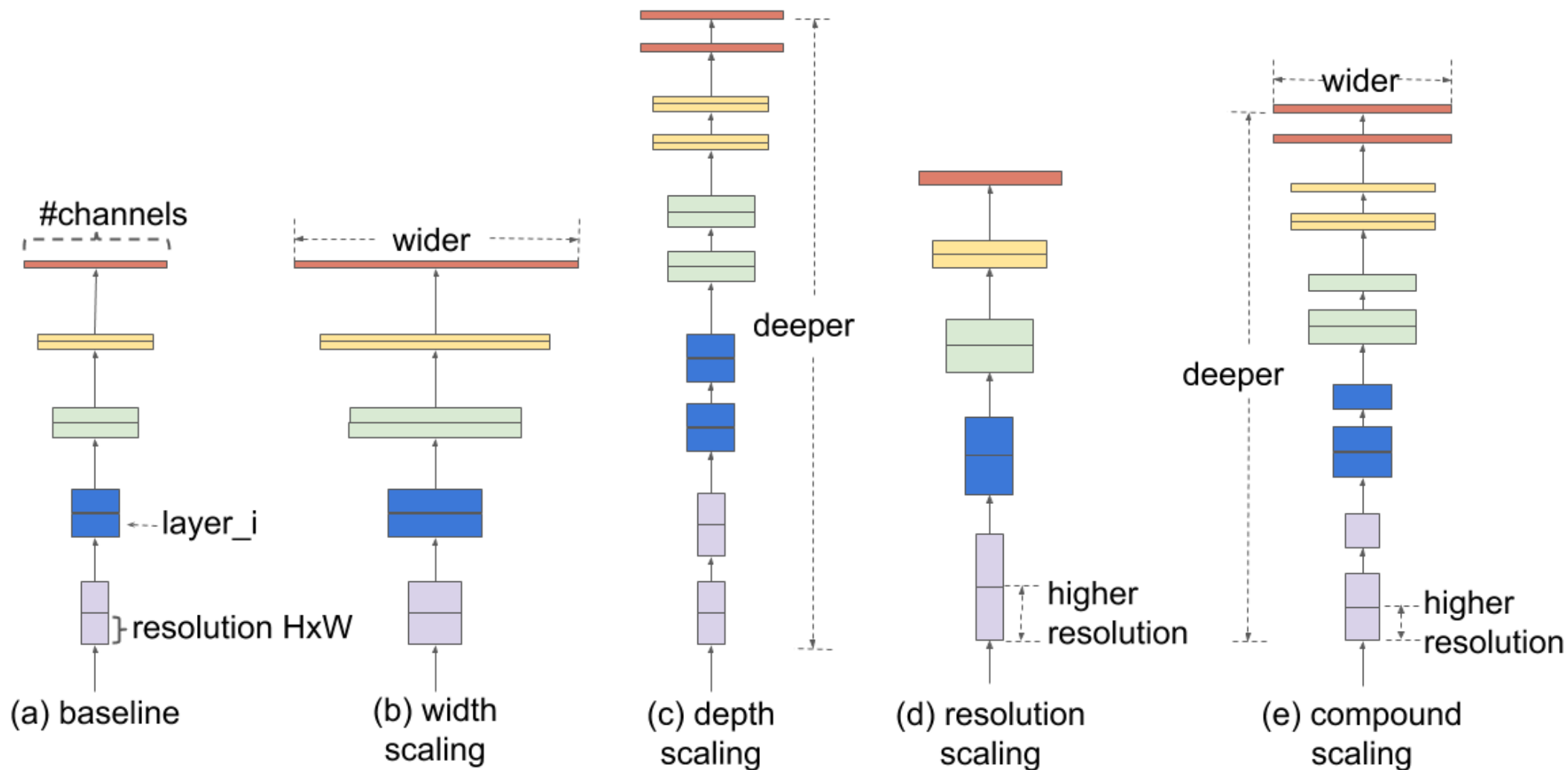
Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
5× Conv dw / s1 Conv / s1	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
	$1 \times 1 \times 512 \times 512$	$14 \times 14 \times 512$
Conv dw / s2	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
Conv / s1	$1 \times 1 \times 512 \times 1024$	$7 \times 7 \times 512$
Conv dw / s2	$3 \times 3 \times 1024$ dw	$7 \times 7 \times 1024$
Conv / s1	$1 \times 1 \times 1024 \times 1024$	$7 \times 7 \times 1024$
Avg Pool / s1	Pool 7×7	$7 \times 7 \times 1024$
FC / s1	1024×1000	$1 \times 1 \times 1024$
Softmax / s1	Classifier	$1 \times 1 \times 1000$

Семейство EfficientNet (2019)

Семейство EfficientNet (2019)

- Обычно свёрточные нейронные сети можно расширять по трем измерениям:
 - Количество слоев в сети (depth)
 - Количество фильтров в свёрточном слое (width)
 - Разрешение исходного изображения (resolution)
- Ключевая идея EfficientNet: расширение свёрточных сетей по трем измерениям с учетом имеющихся вычислительных ресурсов с помощью... другой нейронной сети!
- Семейство состоит из архитектур EfficientNet с различными коэффициентами масштабирования для глубины, ширины и разрешения: EfficientNet-B0, EfficientNet-B1, ..., EfficientNet-B7.

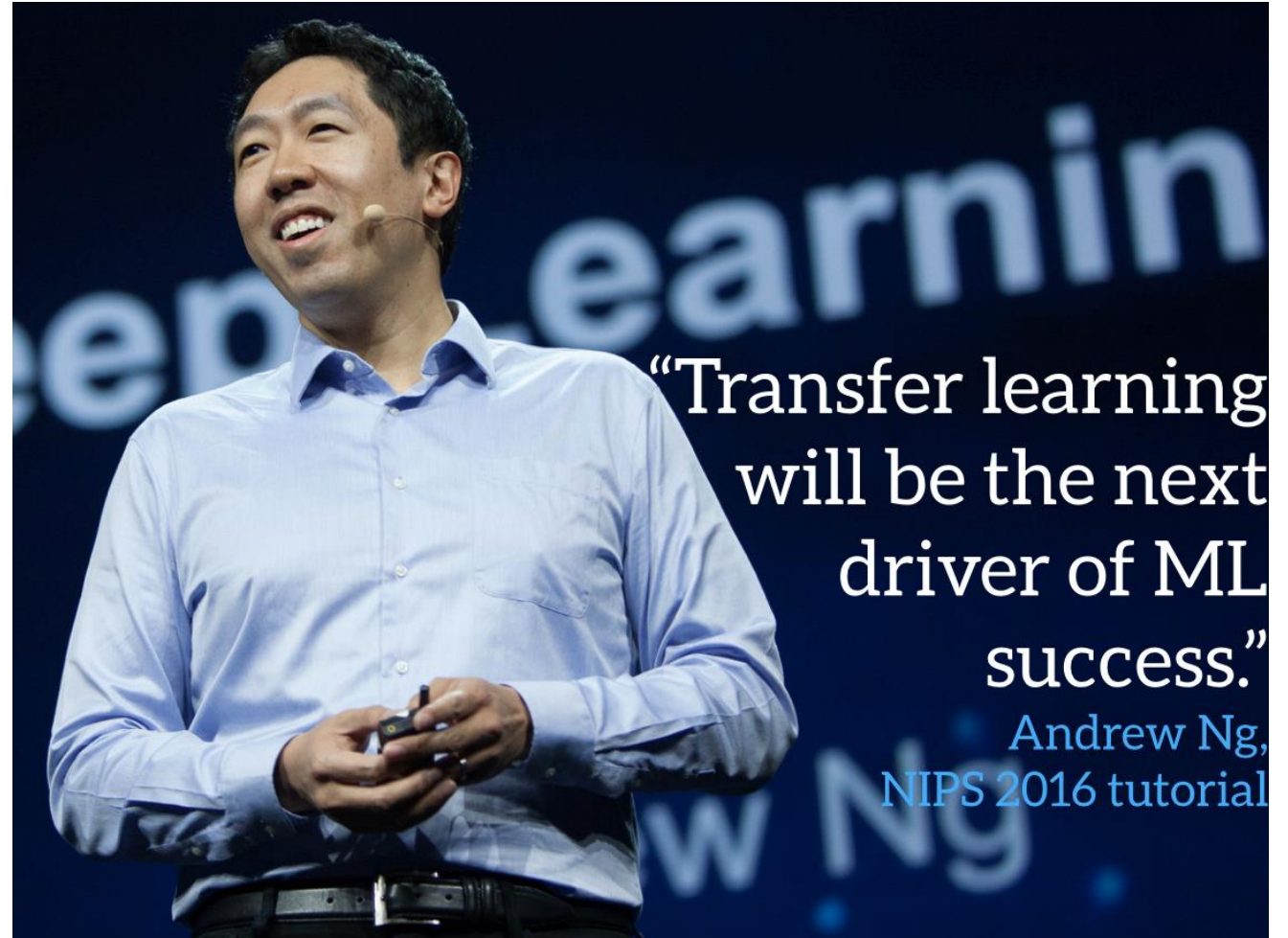
Семейство EfficientNet (2019)



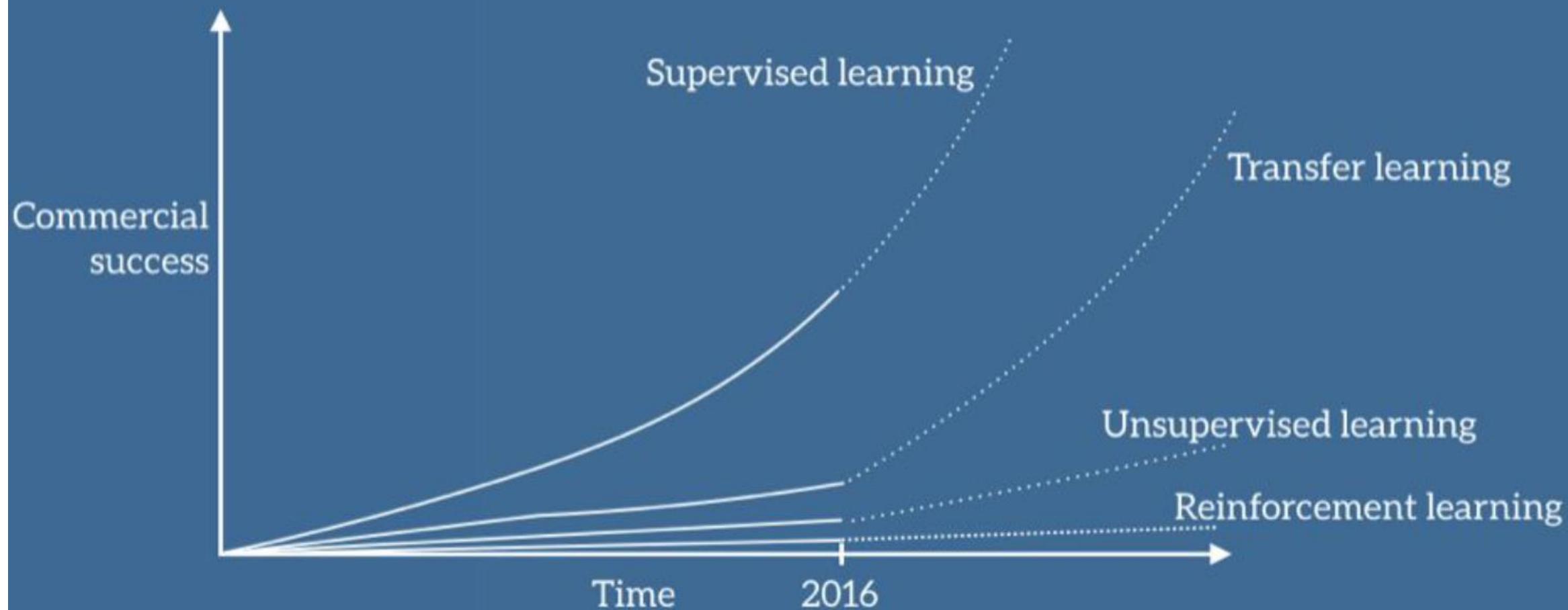
Transfer learning

Transfer learning

- Подход transfer learning — или перенос знаний — является одним из ключевых столпов современных методов анализа данных и машинного обучения в разделе глубинного обучения.



Drivers of ML success in industry



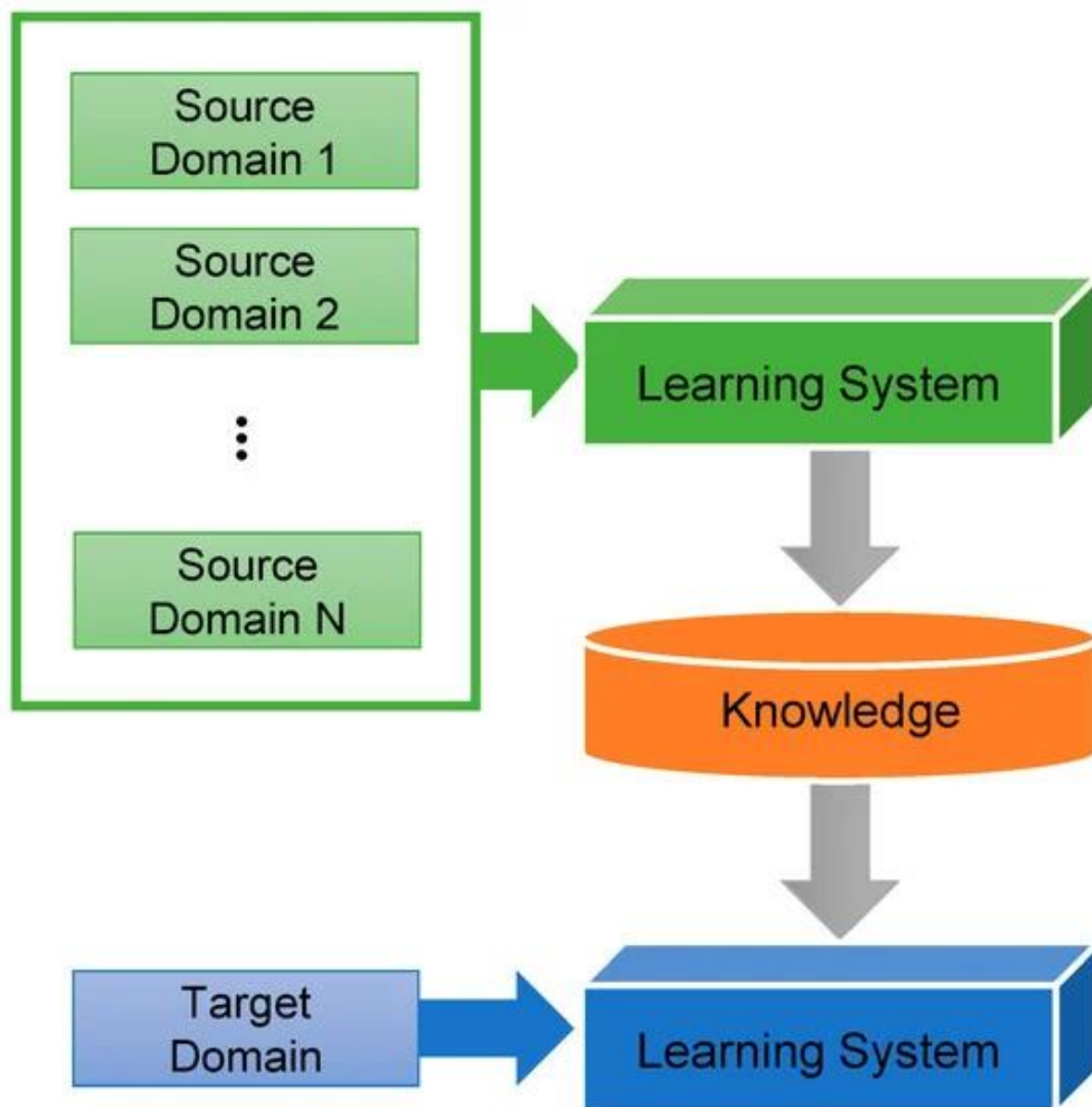
- Andrew Ng, NIPS 2016 tutorial

Transfer learning

- В чем же заключается суть этого подхода?
- Часто датасет под какую-либо задачу для обучения сети содержит мало объектов. И если обучать сеть на этом датасете с нуля, сеть переобучится.
- Примеры:
 - медицинские датасеты изображений опухолей,
 - машинный перевод с малораспространенных языков (татарский)
 - и многое другое.

Transfer learning

- В чем же заключается суть этого подхода?
- Часто датасет под какую-либо задачу для обучения сети содержит мало объектов. И если обучать сеть на этом датасете с нуля, сеть переобучится.
- Примеры:
 - медицинские датасеты изображений опухолей,
 - машинный перевод с малораспространенных языков (татарский)
 - и многое другое.
- Идея: а что, если попробовать использовать знания, полученные другими сетями на похожих задачах!



Transfer learning

- В рамках transfer learning может быть выделено несколько подходов.

Transfer learning

- В рамках transfer learning может быть выделено несколько подходов.
- Первый и самый общий из них — Develop model — подразумевает следующий алгоритм действий:
 1. Выбираем исходную задачу, которая похожа на нашу и для которой есть много данных и разметка (ImageNet).

Transfer learning

- В рамках transfer learning может быть выделено несколько подходов.
- Первый и самый общий из них — Develop model — подразумевает следующий алгоритм действий:
 1. Выбираем исходную задачу, которая похожа на нашу и для которой есть много данных и разметка (ImageNet).
 2. Обучаем модель на этих данных.

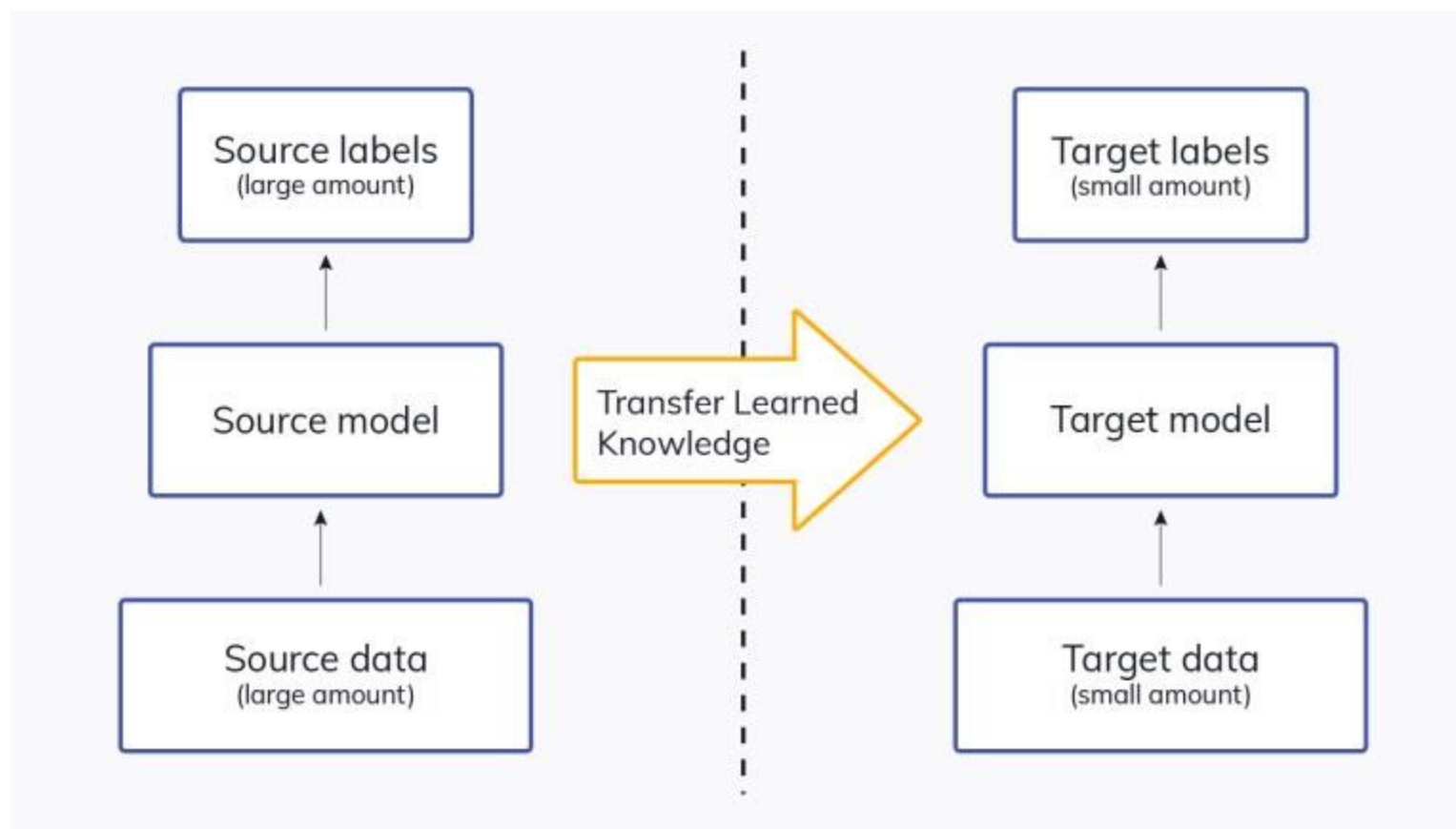
Transfer learning

- В рамках transfer learning может быть выделено несколько подходов.
- Первый и самый общий из них — Develop model — подразумевает следующий алгоритм действий:
 1. Выбираем исходную задачу, которая похожа на нашу и для которой есть много данных и разметка (ImageNet).
 2. Обучаем модель на этих данных.
 3. Адаптируем архитектуру обученной модели для нашей задачи.

Transfer learning

- В рамках transfer learning может быть выделено несколько подходов.
- Первый и самый общий из них — Develop model — подразумевает следующий алгоритм действий:
 1. Выбираем исходную задачу, которая похожа на нашу и для которой есть много данных и разметка (ImageNet).
 2. Обучаем модель на этих данных.
 3. Адаптируем архитектуру обученной модели для нашей задачи.
 4. Дообучаем (fine-tuning) получившуюся модель на датасете для целевой задачи.

Transfer learning



Подход Develop model

Transfer learning

- На практике сверточные сети с нуля обучают только крупные компании из-за серьезной ограниченности ресурсов.

Transfer learning

- На практике сверточные сети с нуля обучают только крупные компании из-за серьезной ограниченности ресурсов.
- Воспользуемся другим подходом — Pre-trained model:
 1. Выбираем уже предобученную за нас модель (на ImageNet).

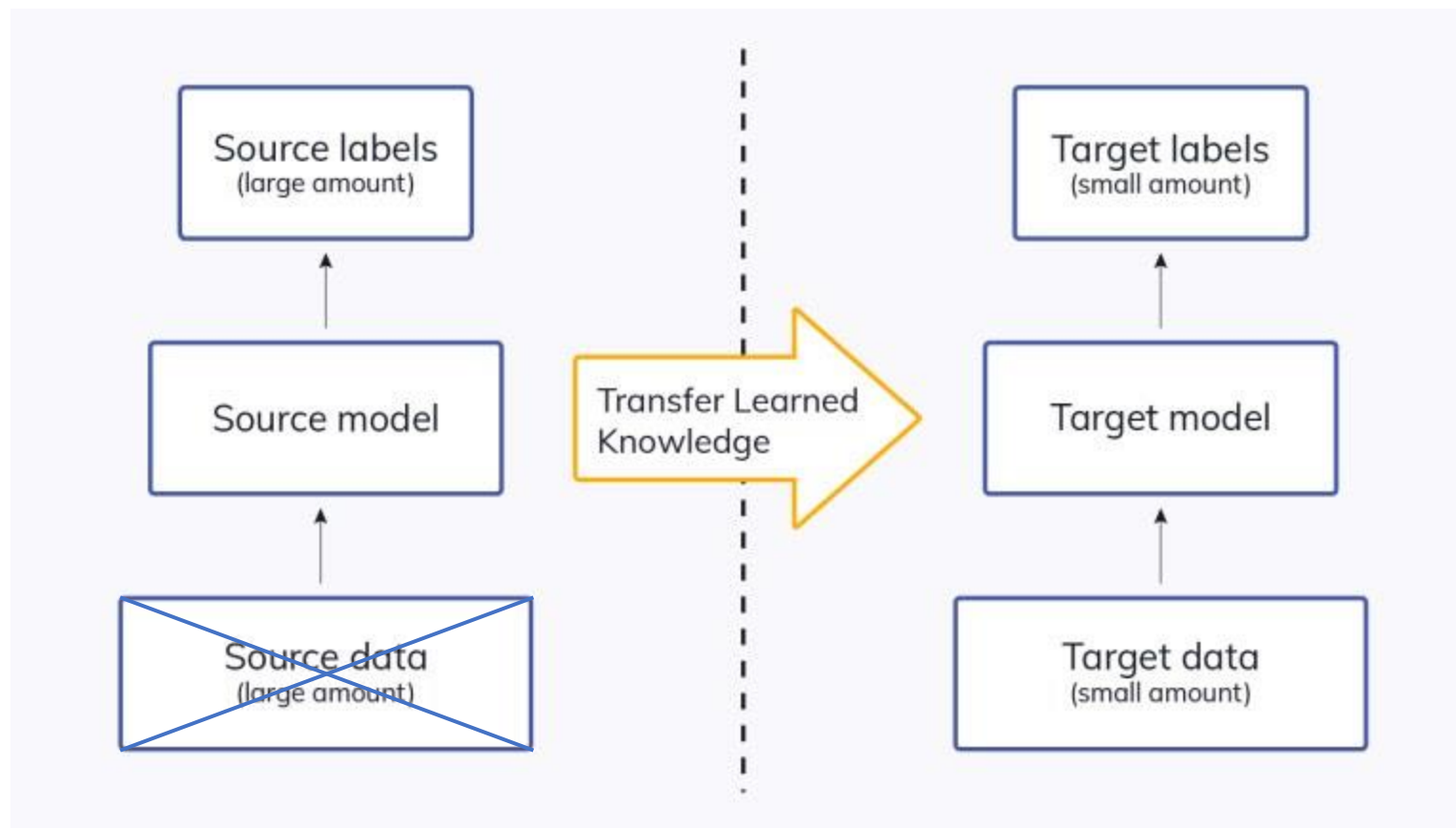
Transfer learning

- На практике сверточные сети с нуля обучают только крупные компании из-за серьезной ограниченности ресурсов.
- Воспользуемся другим подходом — Pre-trained model:
 1. Выбираем уже предобученную за нас модель (на ImageNet).
 2. Адаптируем архитектуру предобученной модели для нашей задачи.

Transfer learning

- На практике сверточные сети с нуля обучают только крупные компании из-за серьезной ограниченности ресурсов.
- Воспользуемся другим подходом — Pre-trained model:
 1. Выбираем уже предобученную за нас модель (на ImageNet).
 2. Адаптируем архитектуру предобученной модели для нашей задачи.
 3. Дообучаем (fine-tuning) получившуюся модель на датасете для целевой задачи.

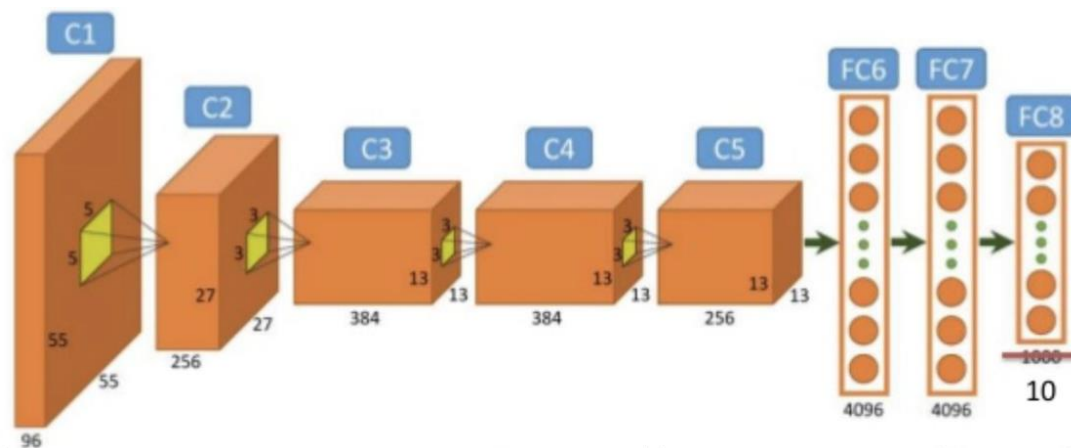
Transfer learning



Подход Pre-trained model

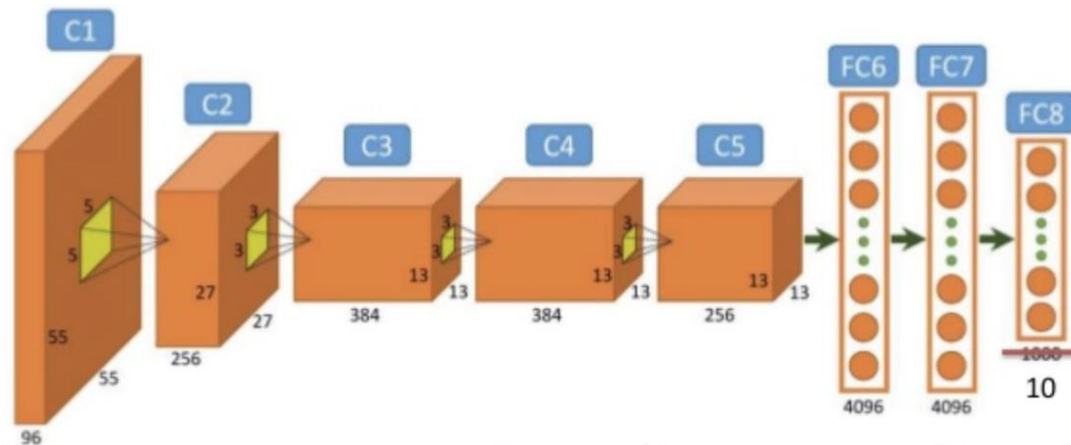
Transfer learning

- Разберемся с последним подходом чуть детальнее.



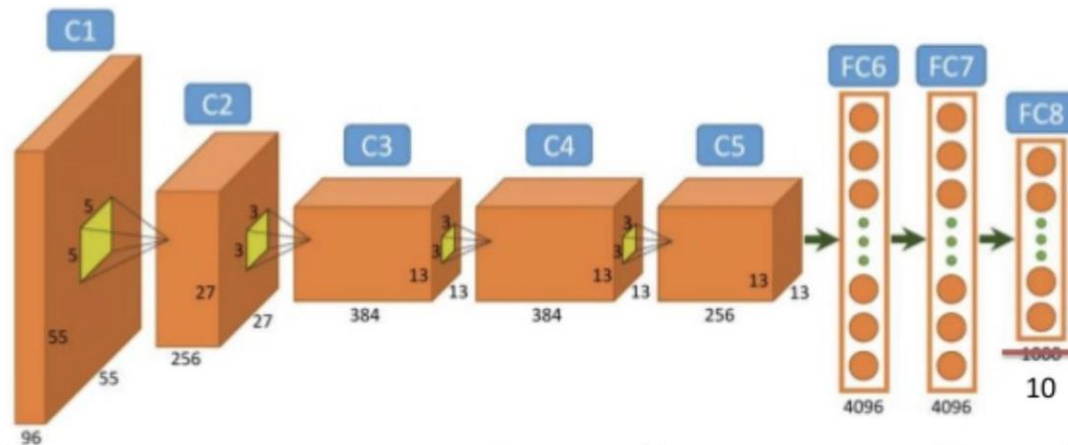
Transfer learning

- Разберемся с последним подходом чуть детальнее.
- В рамках него тоже присутствует вариативность в осуществлении различных действий.



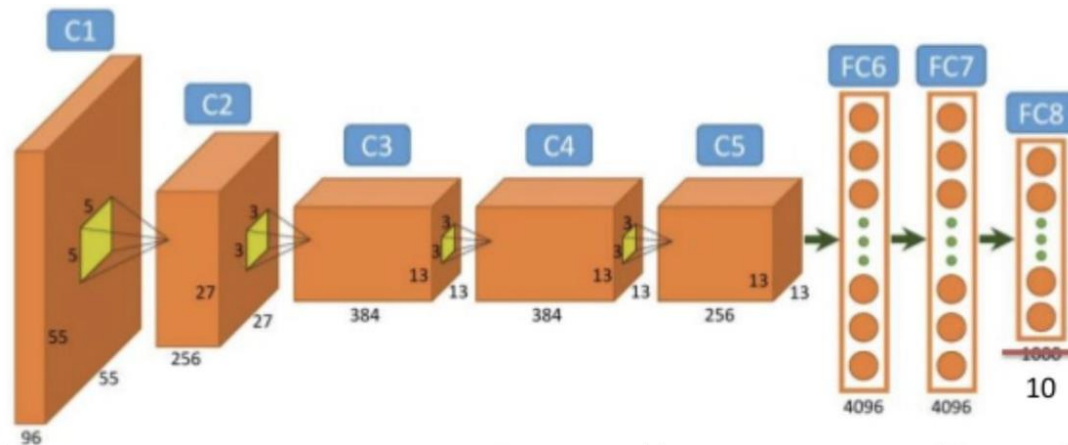
Transfer learning

- Замена классификационного выходного слоя предобученной модели:



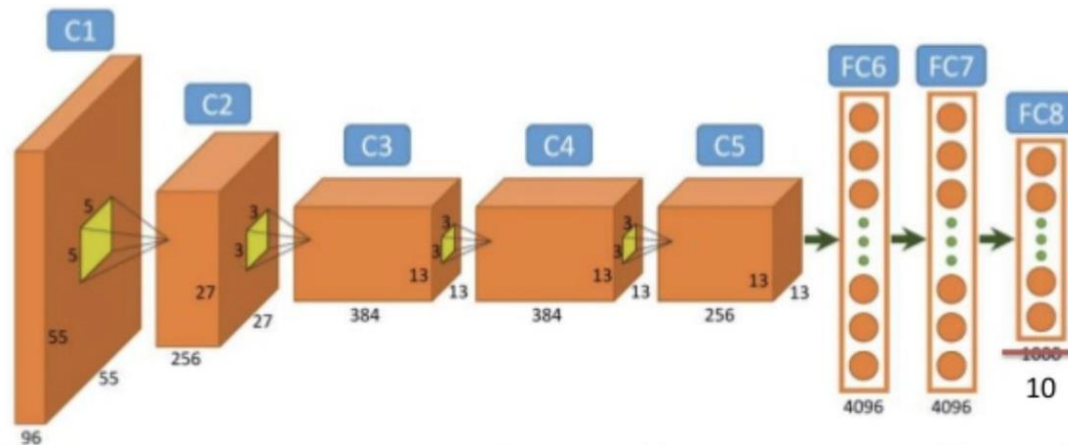
Transfer learning

- Замена классификационного выходного слоя предобученной модели:
 - У модели, предобученной на ImageNet, выходной слой содержит 1000 нейронов.



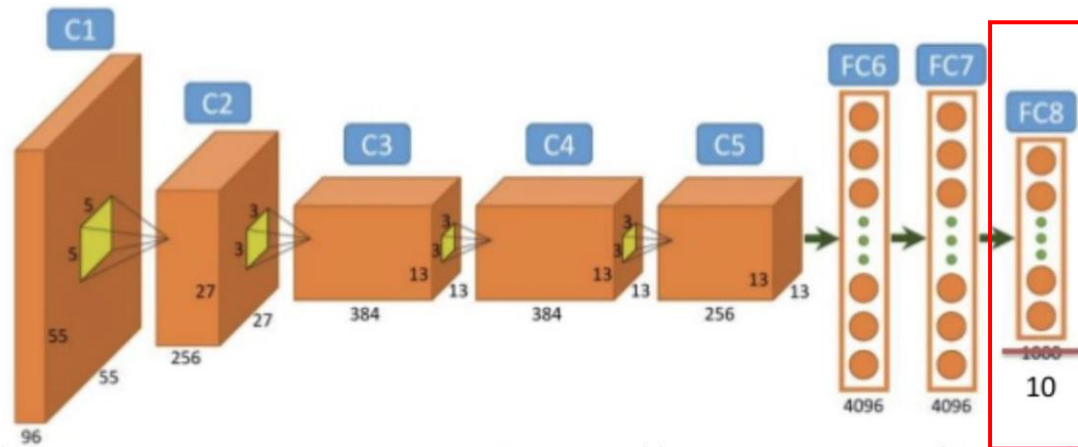
Transfer learning

- Замена классификационного выходного слоя предобученной модели:
 - У модели, предобученной на ImageNet, выходной слой содержит 1000 нейронов. Уберем этот слой.



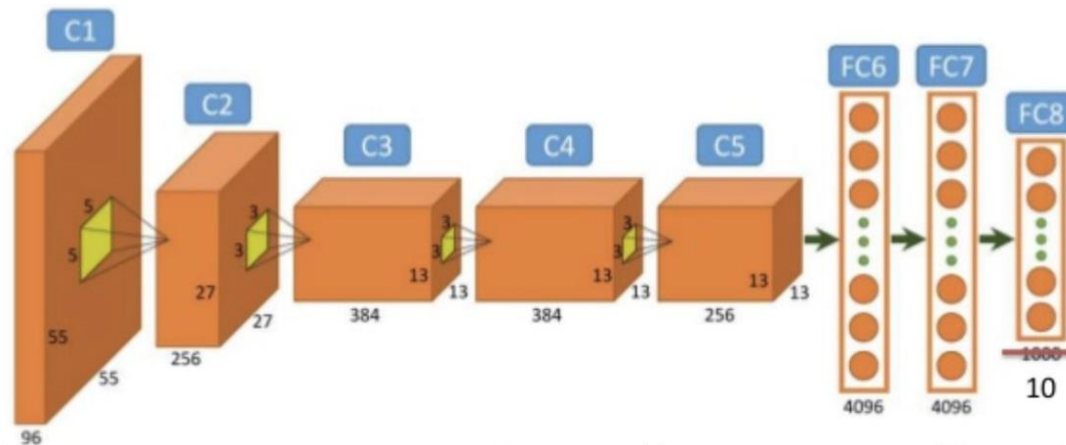
Transfer learning

- Замена классификационного выходного слоя предобученной модели:
 - У модели, предобученной на ImageNet, выходной слой содержит 1000 нейронов. Уберем этот слой.
 - Подставим новый слой с тем количеством нейронов, которое соответствует количеству классов на нашей целевой задаче.



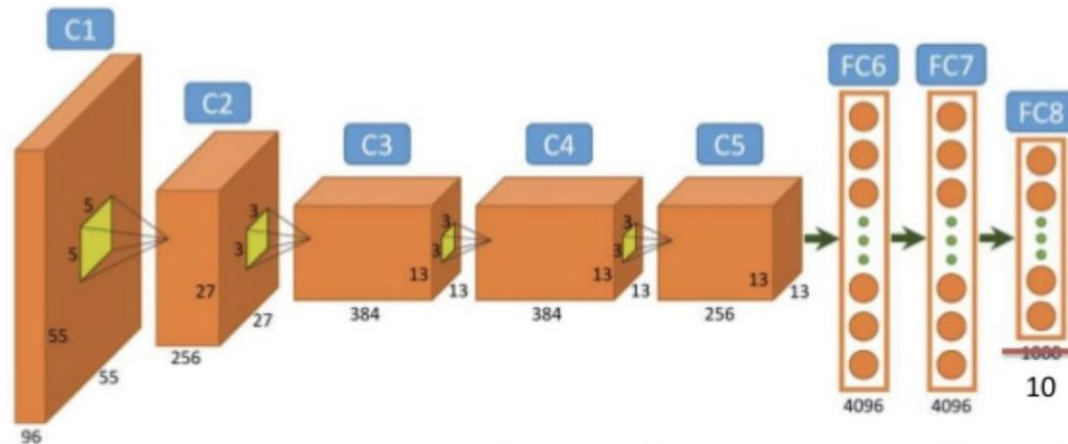
Transfer learning

- Мы также можем принять решение об:
 - удалении некоторых слоев предобученной модели;



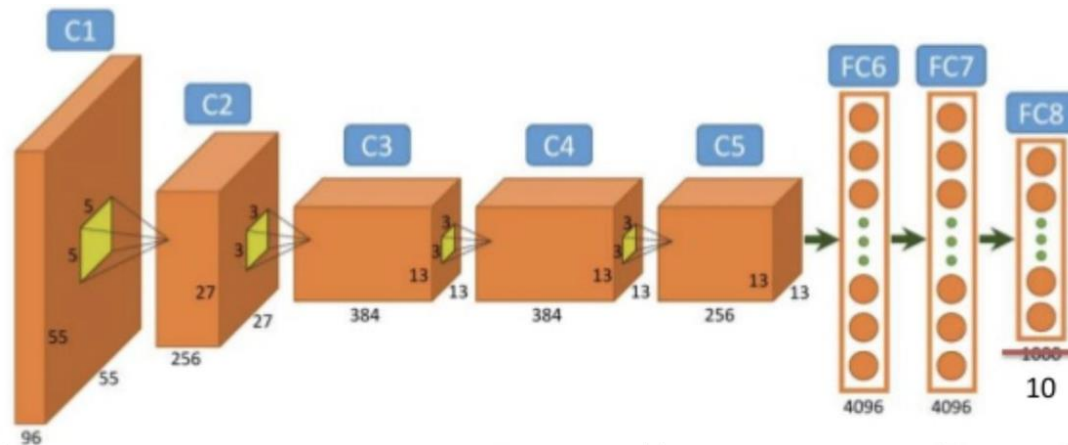
Transfer learning

- Мы также можем принять решение об:
 - удалении некоторых слоев предобученной модели;
 - замене некоторых слоев предобученной модели на другие;



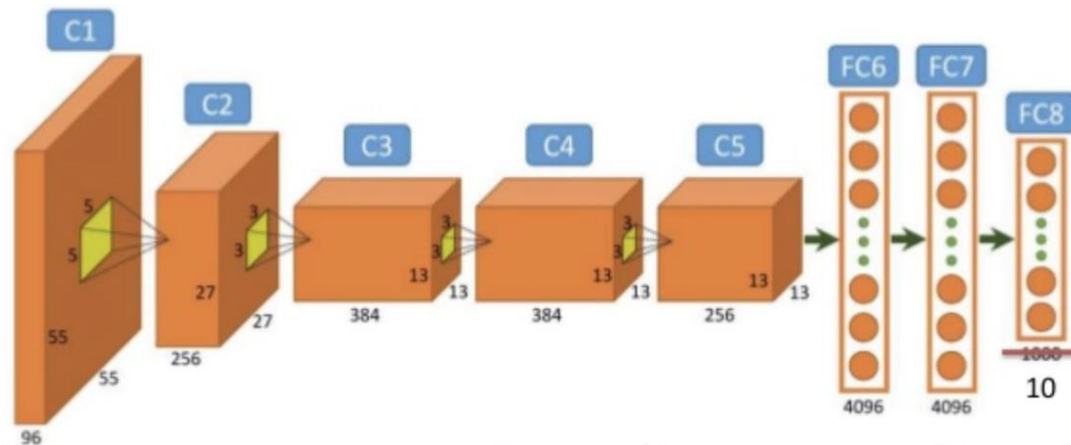
Transfer learning

- Мы также можем принять решение об:
 - удалении некоторых слоев предобученной модели;
 - замене некоторых слоев предобученной модели на другие;
 - добавлении новых слоев.



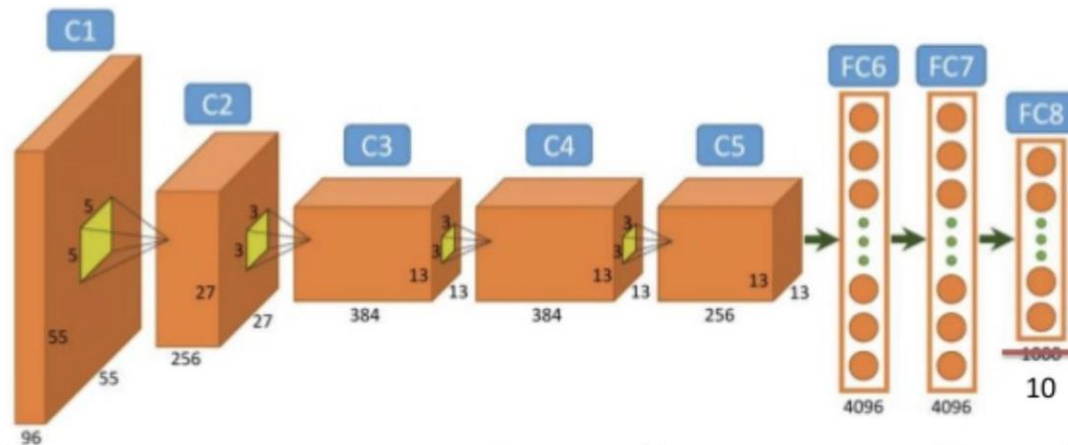
Transfer learning

- Мы также можем принять решение об:
 - удалении некоторых слоев предобученной модели;
 - замене некоторых слоев предобученной модели на другие;
 - добавлении новых слоев.
- Например, слои FC6 и FC7 из примера можем убрать и заменить их слоями другой размерности или с другими характеристиками.



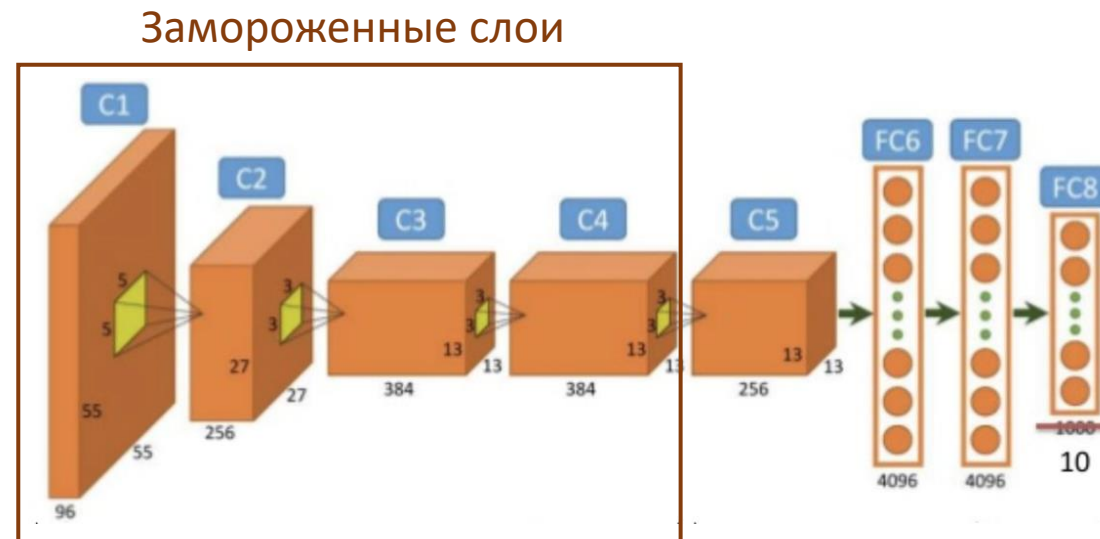
Transfer learning

- Наконец, сам fine-tuning (дообучение) будет осуществляться нами следующим образом:



Transfer learning

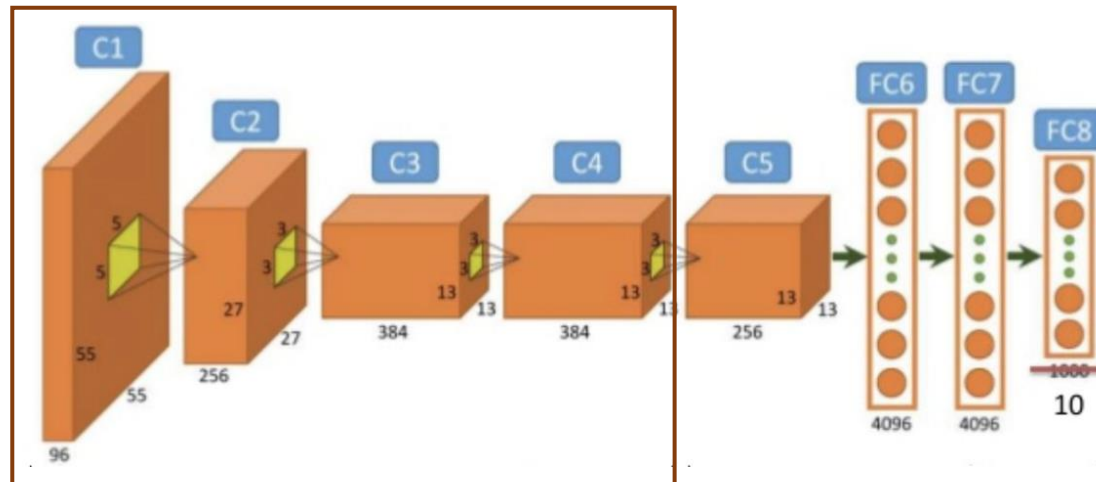
- Наконец, сам fine-tuning (дообучение) будет осуществляться нами следующим образом:
 - Замораживаем первые слои, отвечающие за более общие признаки. Эти признаки пригодятся для решения целевой задачи.



Transfer learning

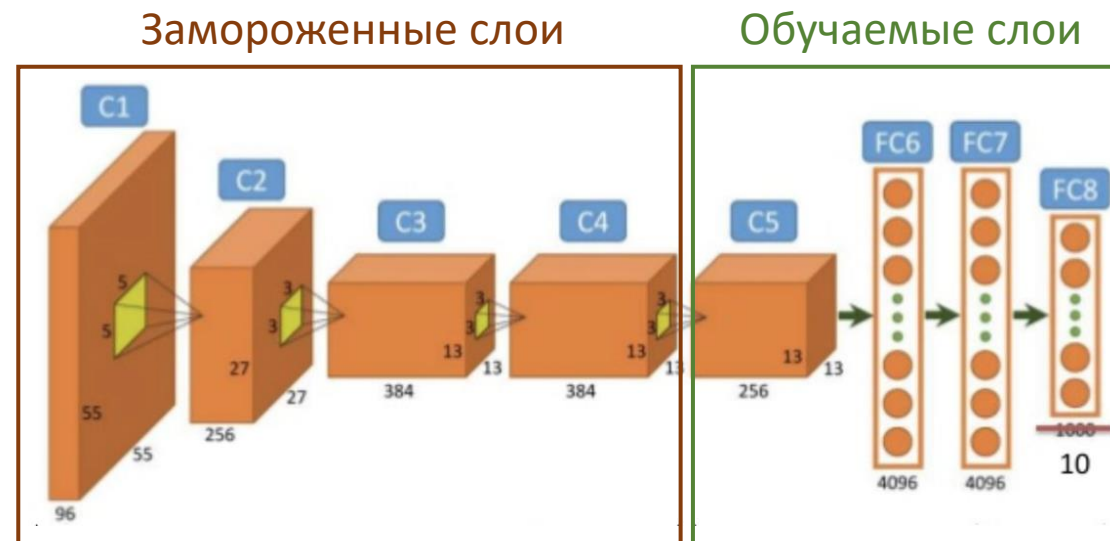
- Наконец, сам fine-tuning (дообучение) будет осуществляться нами следующим образом:
 - Замораживаем первые слои, отвечающие за более общие признаки. Эти признаки пригодятся для решения целевой задачи.
 - Мы также можем принять решение, что не будем замораживать какие-либо из слоёв.

Замороженные слои



Transfer learning

- Наконец, сам fine-tuning (дообучение) будет осуществляться нами следующим образом:
 - Размораживаем и дообучаем более верхние слои. Эти слои содержат специфическую для датасета из исходной задачи информацию, которая на новом датасете не пригодится.



Transfer learning

